30/03/2026

enferVision

LIMS Bulk PRF Data Extraction Process Implemented as Local Web App Using AI Process

Daniel Cullen
20027857

Declaration

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of ‘single quotation marks’, for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Signature:  Date: 30/03/2026

Titles

Business Title

enferVision

Academic Title

LIMS Bulk PRF Data Extraction Process Implemented as Local Web App Using AI Process

Abstract

Patient request forms (PRFs) which detail patient demographics, sample types and requested diagnostic tests are submitted to clinical testing laboratories. These PRFs require manual data entry into laboratory information management systems (LIMS). This manual entry process is time-consuming and susceptible to transcription errors which can in turn lead to downstream processing delays and result errors. The aim of this project was to develop a web-based system designed to automatically extract key fields from scanned forms and provide a structured workflow for validation and downstream data generation.

Several approaches to automated data extraction were evaluated during development. Initially, the use of document processing services such as mPLUG-DocOwl2 were explored, but hardware and resource limitations prevented deployment. As an alternative, traditional optical character recognition (OCR) solutions were explored. An implementation using Tesseract OCR was unable to reliably process PRFs due to skewed labels and inconsistent formatting. PaddleOCR provided improved performance, particularly with skewing, but the data parsing requirements to convert the extracted data into structured fields made this approach complex with high levels of maintenance.

To address these limitations, the system adopted a full AI-based API extraction approach, enabling scanned PRF images to be processed directly and returned as structured JSON data conforming to a predefined schema. This reduced the need for complex parsing while providing confidence scores for each field. The extracted data is imported into a full-stack web application built using Node.js, Hapi.js, MongoDB, and Handlebars, where users can review and edit the extracted information, add observations, and confirm records. Upon confirmation, the system generates a structured XML output and locks the record.

The resulting application provides a semi-automated workflow for PRF processing that combines AI-based extraction with user validation. While the API-based approach significantly improves extraction reliability and implementation simplicity, it does introduce potential privacy considerations due to the transmission of sensitive patient information to an external service. These concerns are currently under evaluation as part of the ongoing development of the application.

Table of Contents

Table of Figures

1. Introduction

1.1 Background

Patient request forms (PRFs) which detail patient demographics, sample types and requested diagnostic tests are submitted by general practitioners (GPs) to clinical testing laboratories. Subsequently, these PRFs require manual data entry into laboratory information management systems (LIMS). This manual entry process is time-consuming and susceptible to transcription errors which can in turn lead to downstream processing delays and result errors.

Moreover, while the existing process has been optimised to support manual data entry, further performance gains can only be achieved via horizontal scaling which requires increases in staffing, relevant equipment and physical infrastructure resulting in significant operational costs. Such scaling would also need to be proportional to sample volumes. Thus, as sample volumes continue to increase, scaling and therefore operational costs, would also need to continually increase.

The introduction of an automated application capable of ingesting and processing bulk-scanned PRFs, extracting the relevant information and generating a file in a format for ingestion and downstream processing by an existing application that applies the necessary data validation rules would facilitate vertical scaling of the process workflow. The resulting workflow would allow for faster order registration with a reduction in the occurrence of human error and increase the overall processing capacity of the sample receipt laboratory. Thus, a higher volume of samples could be registered in a shorter amount of time without incurring significant financial and operational costs.

1.2 Problem Definition

The existing process for logging GP patient orders into a LIMS relies on manual data entry which limits scalability, introduces errors and increases operational costs. This approach cannot meet demands as sample volumes increase without a proportional and costly growth in resources. An automated process for extracting PRF data is required to improve order throughput and allow for cost-effective scalability.

1.3 Proposed Solution

To solve the defined problem, a locally hosted browser-based app which applies an AI-based document processing service is proposed. Briefly, pre-printed barcodes containing unique IDs are applied to each PRF which are then bulk scanned. The resulting files are ingested by the application. The AI-based document processing service extracts the required information from the PRF and stores it as a unique record. The user logs into the application and scans the physical PRF into a search field to return the record and its associated details. The user reviews the data, makes amendments if necessary and confirms the order. On confirmation, a formatted file is created containing the necessary data for downstream ingestion.

1.4 Project Goals

The following key goals are defined for the implementation of the new process workflow:

1) Build secure locally hosted browser-based application capable of ingesting GP PRFs in bulk and storing each as a unique record.
2) Apply an AI-based document processing service to automatically extract the relevant details from each PRF.
3) Build an interface that allows users to edit and confirm the extracted data.
4) Add functionality to transform the confirmed data into an XML formatted file.

The following stretch goals are defined for the implementation of the new process workflow:

1) Provide secure authentication and basic security measures (hashing, salting, sanitisation etc.).
2) Provide basic auditability and administrative oversight.
3) Show reliability with unit tests and Cypress.

2. Research and Analysis

2.1 Data Characteristics and Constraints

For the current application, scanned PRFs are the primary source of data. The majority of PRFs submitted are well-structured forms containing mainly typed information as shown in Figure 1. AI-based document processing services have been shown to be effective in extracting and interpreting data from such structured forms (Ke *et el*., 2025; Mahadevkar *et al*., 2024; Bajrami *et al.,* 2023; Beerten, 2023; Shaikh *et al*., 2023; Kim *et al*., 2022).

ST. JAMES'S HOSPITAL

St. James's Hospital HOPe Directorate

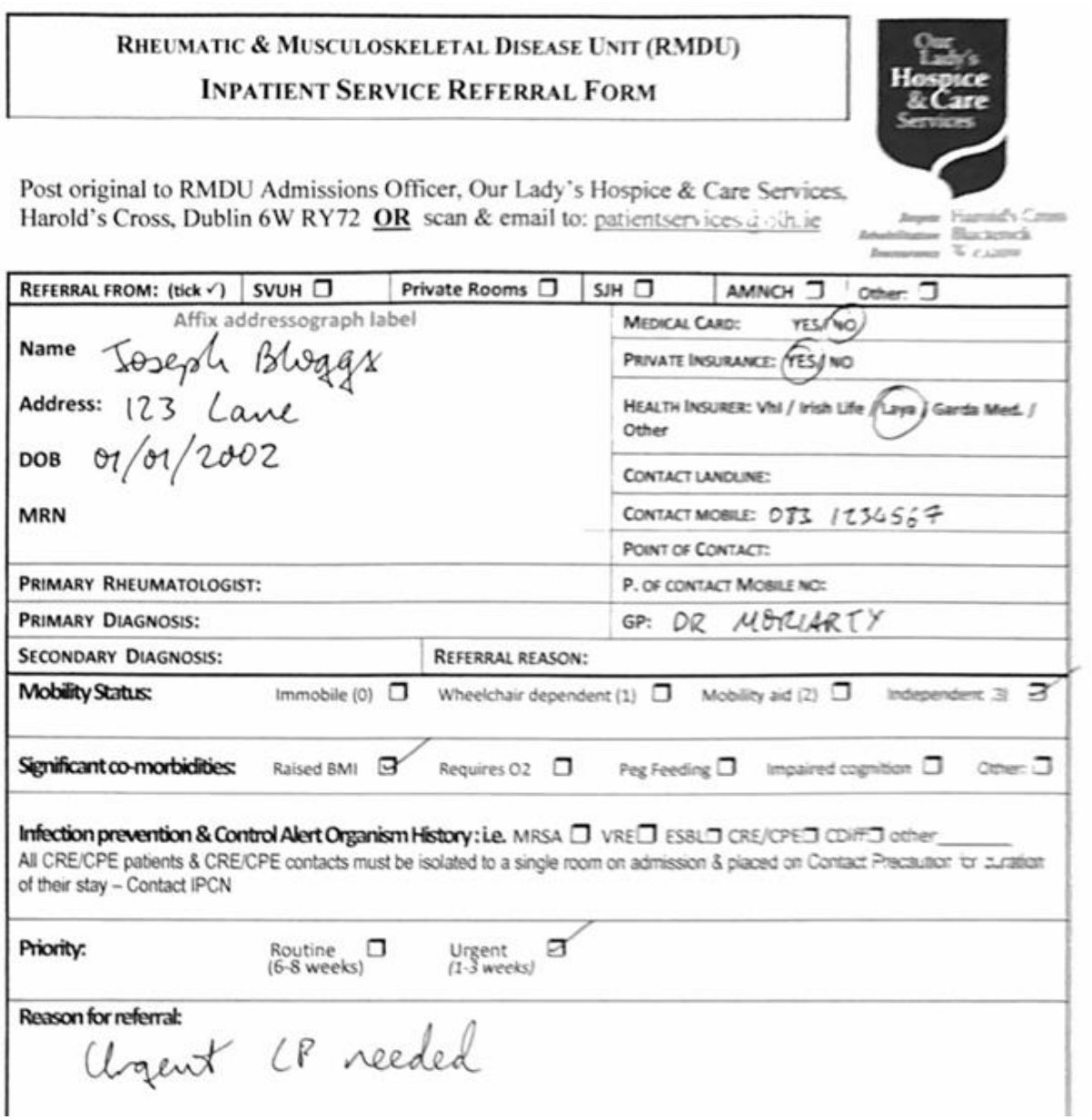
Patient Referral Form for Lymphoma MDT

Document Number	MF-SCT-0012	**Revision Number**	3	**Effective Date**	20/01/2022
Owner:	Quality Manager			**Approved by:**	Prof E. Vandenberghe

Patient Details	
Patient Name: Joe Bloggs **Address:** 15 Peach Trees, Mega City One	**Date of Birth:** 01/01/1955
General Practitioner Details	
Name: Dr Caligari	
Address: St Bartholomew's Hospital	

Figure 1: Example of a structured data PRF from St. James's Hospital (St James's Hospital, 2026).

However, as shown in Figure 2, a subset of forms received, while well-structured, also contain handwritten segments that require extraction and interpretation. Current AI-based document processing services have been shown to be generally less effective when handling handwritten content, thereby necessitating the need for more advanced extraction and interpretation techniques involving handwriting-capable services such as Google Cloud Vision or Google Gemini (Makashyn, 2025; Terzo, 2024; Sharma, 2023).



RHEUMATIC & MUSCULOSKELETAL DISEASE UNIT (RMDU)

INPATIENT SERVICE REFERRAL FORM

Our Lady's Hospice & Care Services

Post original to RMDU Admissions Officer, Our Lady's Hospice & Care Services, Harold's Cross, Dublin 6W RY72 **OR** scan & email to: patientservices...ie

REFERRAL FROM: (tick ✓) SVUH ☐ Private Rooms ☐ SJH ☐ AMNCH ☐ Other: ☐

Affix addressograph label

Name Joseph Bloggs

Address: 123 Lane

DOB 01/01/2002

MRN

MEDICAL CARD: YES / NO

PRIVATE INSURANCE: YES / NO

HEALTH INSURER: VHI / Irish Life / Laya / Garda Med. / Other

CONTACT LANDLINE:

CONTACT MOBILE: 083 1234567

POINT OF CONTACT:

P. OF CONTACT MOBILE NO:

GP: DR MORIARTY

PRIMARY RHEUMATOLOGIST:

PRIMARY DIAGNOSIS:

SECONDARY DIAGNOSIS:

REFERRAL REASON:

Mobility Status: Immobile (0) ☐ Wheelchair dependent (1) ☐ Mobility aid (2) ☐ Independent (3) ☒

Significant co-morbidities: Raised BMI ☒ Requires O2 ☐ Peg Feeding ☐ Impaired cognition ☐ Other: ☐

Infection prevention & Control Alert Organism History: i.e. MRSA ☐ VRE ☐ ESBL ☐ CRE/CPE ☐ CDiff ☐ other _____

All CRE/CPE patients & CRE/CPE contacts must be isolated to a single room on admission & placed on Contact Precaution for duration of their stay – Contact IPCN

Priority: Routine (6-8 weeks) ☐ Urgent (1-3 weeks) ☑

Reason for referral:

Urgent LP needed

Figure 2: Example of a structured data PRF with Handwritten elements (Our Lady's Hospice & Care Services, 2026).

Due to the sensitive nature of the information contained within PRFs however, including personally identifiable information, handwritten segments should first be detected and isolated. This enables selective cropping of the document, ensuring that only the handwritten segments are transmitted for external processing, thereby minimising exposure of identifiable data.

2.2 Evaluation of Document Processing Approaches

As discussed, the majority of received PRFs contain structured data that currently requires manual entry into a LIMS. Optical character recognition (OCR) technologies and AI-based document intelligence services both provide solutions for the current problem. OCR software, such as Tesseract or PaddleOCR, applies algorithms to recognise and extract characters in scanned images and translate them into machine-readable text (Francis & Sangeetha, 2025).

Traditionally, OCR systems used rule-based algorithms, applying machine learning models such as support vector machines and k-nearest neighbours, which resulted in limited accuracy and required a significant amount of human intervention (Francis & Sangeetha, 2025). OCR software has seen significant performance improvements in recent years however, with the application of deep learning models such as recurrent neural networks and convolutional neural networks (Francis & Sangeetha, 2025; Murugan *et al*., 2025; Singh, Gupta, & Garg, 2022). OCR faces prominent challenges in accurately predicting language however, as a result of handwriting style variation, irregular spacing, overlapping characters and form layout complexity (Francis & Sangeetha, 2025; Murugan *et al*., 2025).

AI-based document processing services have shown greater promise than traditional OCR-based technologies, addressing many of the challenges associated with OCR using combined techniques such as computer vision, deep learning, machine learning, and text analytics (Ke *et el*., 2025; Mahadevkar *et al*., 2024). Such services allow unstructured data to be interpreted and analysed more accurately with minimal human intervention (Ke *et el*., 2025; Mahadevkar *et al*., 2024). AI-based document processing services include OCR-free models such as Donut, Pix2Struct, LayoutLM and mPLUG-Doc/mPLUG-DocOwl2, vision-language models (VLMs) such as Claude and Gemini, document large language models (LLMs) such as DocLLM, InstructDoc and Table-LLM, enterprise commercial AI services such as Azure Document Intelligence, Amazon Textract and Google Document AI, and applications such as ChatDOC, RagFlow and MinerU.

Donut, Pix2Struct, LayoutLM and mPLUG-Doc/mPLUG-DocOwl2 are examples of document processing services that can all be hosted and used locally. Donut is an effective visual document understanding method that has proven effective in document processing, even outperforming LayoutLM and Pix2Struct (Bajrami *et al.,* 2023; Beerten, 2023; Shaikh *et al*., 2023; Kim *et al*., 2022).

However, while the model performs efficiently, it requires significant computational resources as it has over two hundred million parameters (Shaikh *et al*., 2023). Pix2Struct is pre-trained using masked screenshots of webpages, making it less effective in parsing scanned forms (Lee *et al*., 2023). LayoutLM has been shown to be effective for documents with complex layouts but performed less effectively overall compared to Donut (Bajrami *et al.,* 2023).

DocOwl and its variations have been shown to be an effective document processing service however (Hu *et al.,* 2024a; Ye *et al*., 2023). Furthermore, DocOwl has also been shown to outperform Donut and Pix2Struct on document understanding tasks, achieving state-of-the-art OCR-free performance against numerous visual document understanding benchmarks (Hu *et al.,* 2024a; Ye *et al*., 2023). DocOwl also features a shape-adaptive cropping module that can partition document images into sub-sections allowing handwritten sections to be isolated for targeted processing via an API call to a handwriting-capable service while maintaining patient information security (Hu *et al*., 2024a; Hu *et al*., 2024b).

Numerous handwriting-capable service options exist including Google Gemini, Google Cloud Vision, Amazon Textract and Microsoft Azure AI Vision. These services have various strengths and weaknesses. However, services such as Google Cloud Vision, Amazon Textract and Microsoft Azure AI Vision, all apply OCR as the first step in their models followed by post-processing which, as discussed previously, faces significant challenges (AWS, 2026; Google Cloud, 2026; Microsoft, 2026; Francis & Sangeetha, 2025). Google Gemini and OpenAI on the other hand use a multi-modal model where OCR and interpretation occur simultaneously, resulting in a more efficient extraction process and improved accuracy (Gemini API, 2026; Torres, 2025).

2.3 Security, Privacy, and Compliance Considerations

Security, privacy and compliance considerations are based on current company requirements and policies. As the application processes scanned PRFs containing sensitive patient personal and clinical information, security and data protection are a primary concern. To comply with company policies, the current application and all associated data requires deployment behind the company's network firewall. This includes the associated database. In addition, robust authentication and authorisation measures are required to ensure only approved users can access the system and those with access are confined to designated roles.

Furthermore, in relation to application of an external handwriting-capable service as discussed above, the system should aim to only share the necessary data with external providers. Auditability (discussed below), is also a key requirement, providing end-to-end traceability, showing who accessed, modified and/or confirmed the extracted data and when these events occurred. The principle of least privilege will also be applied in order to ensure that users are granted the minimal level of access required.

2.4 Output Requirements

The extracted and confirmed data output by the application will be ingested by a downstream application, where validation rules and sample numbers will be applied. The current application does not exist in isolation and forms a part of a larger process, thus consistent data formats suitable for automated downstream ingestion are required. Numerous structured formatting options exist including JSON, HL7 and XML. JSON is an easy-to-read and lightweight text-based data format used for storing and transporting data (W3 Schools, 2026; GeeksforGeeks, 2025). It is self-describing and stored in key-value pairs and it is widely used for client-server communication (W3 Schools, 2026; GeeksforGeeks, 2025).

HL7 is a set of standards developed for the exchange of health information between applications (HL7 International, 2026; InterfaceWare, 2026; Rhapsody, 2026; HL7 Standards 2015). HL7 messages are composed of over 120 distinct segments such as a message header, patient information and observation request to name a few (HL7 International, 2026; InterfaceWare, 2026; Rhapsody, 2026; HL7 Standards 2015). Custom segments are also possible when using HL7 standards applications (HL7 International, 2026; InterfaceWare, 2026; Rhapsody, 2026; HL7 Standards 2015).

XML is a mark-up language without predefined tags designed to be both human- and machine-readable (Mdn, 2026l W3 Schools, 2026). XML allows a user to define their own tags while maintaining a fundamental format, providing flexibility alongside standardisation (Mdn, 2026; W3 Schools, 2026). XML formatting is preferred for the current application due to the requirements of the downstream application developer.

2.5 Audit and Traceability Requirements

The application is required to operate within the constraints of the company's quality management system (QMS) and in accordance with the expectations of The Irish National Accreditation Board (INAB) accreditation standards. Both the company's QMS and INAB place a strong emphasis on accountability, traceability and the capacity to demonstrate oversight on all data-handling and decision-making processes. Auditability is critical to ensure regulatory compliance and data integrity (SimplerQMS, 2025; LabManager, 2024). Thus, a clear and reliable audit trail is required for PRF processing.

Audit history requirements include recording of PRF ingestion and data extraction, data modification and data confirmation. Except for ingestion and data extraction, events must be attributable to a specific user and timestamped to allow for retrospective review where needed, such as for compliance audits and non-conformance investigations. To further support these requirements, retrieval of the audit history data in a structured and useful manner is a necessity. This level of traceability is essential to demonstrate that extracted data has been subject to appropriate oversight and that responsibility for processing and confirmation can be clearly identified.

2.6 Technology Stack Considerations

Usability, maintainability and deployment requirements inform the choice of technology stack for the application. A browser-based application was chosen over a traditional desktop application as a browser-based application allows system access without the need for local installation or any platform-specific dependencies (Host IT Smart, 2026; Ramotion, 2026; Medium, 2024). Moreover, this approach supports a simplified rollout with centralised updates and aligns with current systems in place within the company (Host IT Smart, 2026; Ramotion, 2026; Medium, 2024). To manage the system data, persistent database storage is required in order to reliably store and retain PRF records, extracted data, user credentials and audit histories.

Furthermore, a database-backed approach allows for structured querying, enforcement of data integrity, and long-term retention in line with company, QMS and accreditation policies. A relational database solution, such as a MySQL database, would provide a fixed schema with highly structured and stable data. The data extracted by AI document processing services can be semi-structured however, particularly as PRFs vary extensively within and between medical practices depending on testing requirements.

File-based storage was considered but rejected as it provides limited support for concurrent access, and querying as well as auditability (AWS, 2026; IBM, 2026; IONOS, 2026). A file-based storage system would make document states and events difficult to reliably track (AWS, 2026; IBM, 2026; IONOS, 2026).

A document-oriented database such as MongoDB, or a relational database with semi-structured support such as PostgreSQL (using JSONB data formatting) provide flexibility in terms of data structure while also allowing for querying and indexing (AWS, 2026; MongoDB, 2026; GeeksforGeeks, 2025). Document-oriented databases, such as MongoDB, are NoSQL databases that store large volumes of semi-structured, or even unstructured, data in flexible, JSON-like documents (AWS, 2026; MongoDB, 2026; GeeksforGeeks, 2025). For the current implementation, this would allow the semi-structured extracted PRF data to be stored and retrieved effectively.

A relational database such as PostgreSQL in conjunction with a JSONB data format, would also allow the storage and extraction of semi-structured data while also allowing a rigid schema-enforced option for structured data such as user info and audit histories (AWS, 2026; MongoDB, 2026; GeeksforGeeks, 2025). Databases such as MongoDB however, are optimised for high read and write performance with large volumes of semi-structured or unstructured data, a key requirement for the current implementation (AWS, 2026; MongoDB, 2026; GeeksforGeeks, 2025). In addition, in comparison to a PostgreSQL JSONB hybrid solution, a MongoDB solution provides more flexibility in handling dynamic, semi-structured or unstructured data while allowing for simpler scaling – another requirement for the current implementation based on company IT infrastructure (AWS, 2026; MongoDB, 2026; GeeksforGeeks, 2025).

Technology stack choice is also influenced by testing requirements which form a significant element of the QMS where testing and validation is critical for deployment of any tool, process or application. A browser-based application with a defined back-end API allows for clear and effective unit and end-to-end testing. This is essential in providing supporting evidence of functionality for validation and quality reports. Testing capabilities are also essential for ongoing maintenance and updates.

3. Modelling

3.1 Process Workflow

As shown in Figure 3, pre-printed barcodes containing a unique identifier are applied to each PRF prior to bulk scanning within the laboratory. The scanned PRFs are deposited into a monitored ingestion folder, where they will be automatically ingested by the application. Each file will be registered as a new PRF record, with the barcode identifier used as the primary identifier for subsequent retrieval and processing.

An AI-based document processing service will analyse the scanned PRF and extract the required data, including the barcode identifier and any associated request information. Handwritten elements identified by the document processing service will be bound and cropped before being included in an API call, along with the unique identifier, to a handwriting-capable service for analysis and extraction. The extracted data will be stored as a unique record linked to the original scanned document.

Authorised users will access the system through a secure login and retrieve PRF records by scanning the physical barcode into a search field within the application. Users will review the extracted data alongside the scanned PRF image, making any necessary amendments to address low confidence or ambiguous fields, before confirming the request once the extracted data has been verified. Upon confirmation, the PRF record will be locked to prevent further modification, and a formatted output file will be generated for downstream application ingestion.

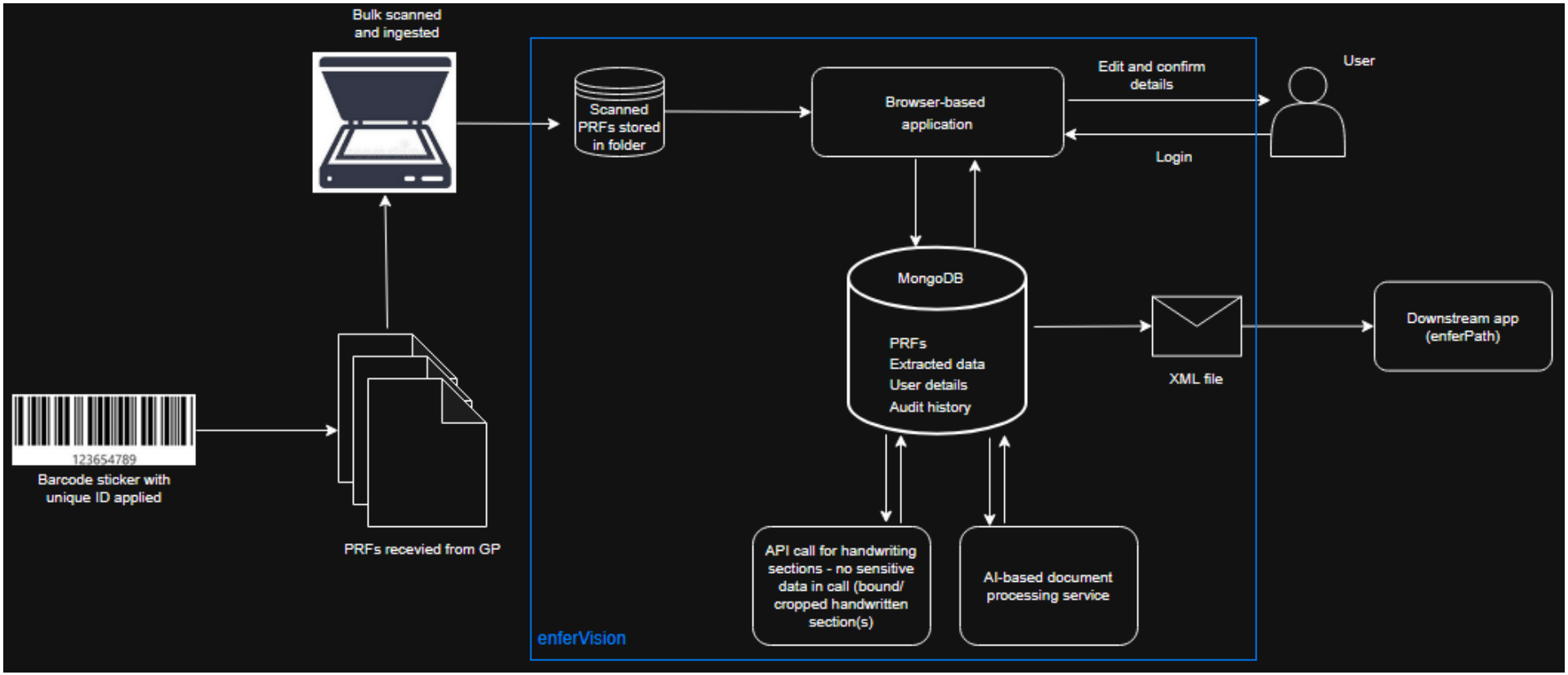


Figure 3: Proposed new process workflow for PRF data extraction and processing.

3.2 Front-end

The front-end will be implemented using HTML, CSS (custom and Bulma) and JavaScript, with Handlebars used for server-side templating. On opening the application, users will be presented with a login screen. Figure 4 provides a mock-up of this login screen. A standard login, for regular users, and a Microsoft OAuth option, for non-regular users, will be provided.

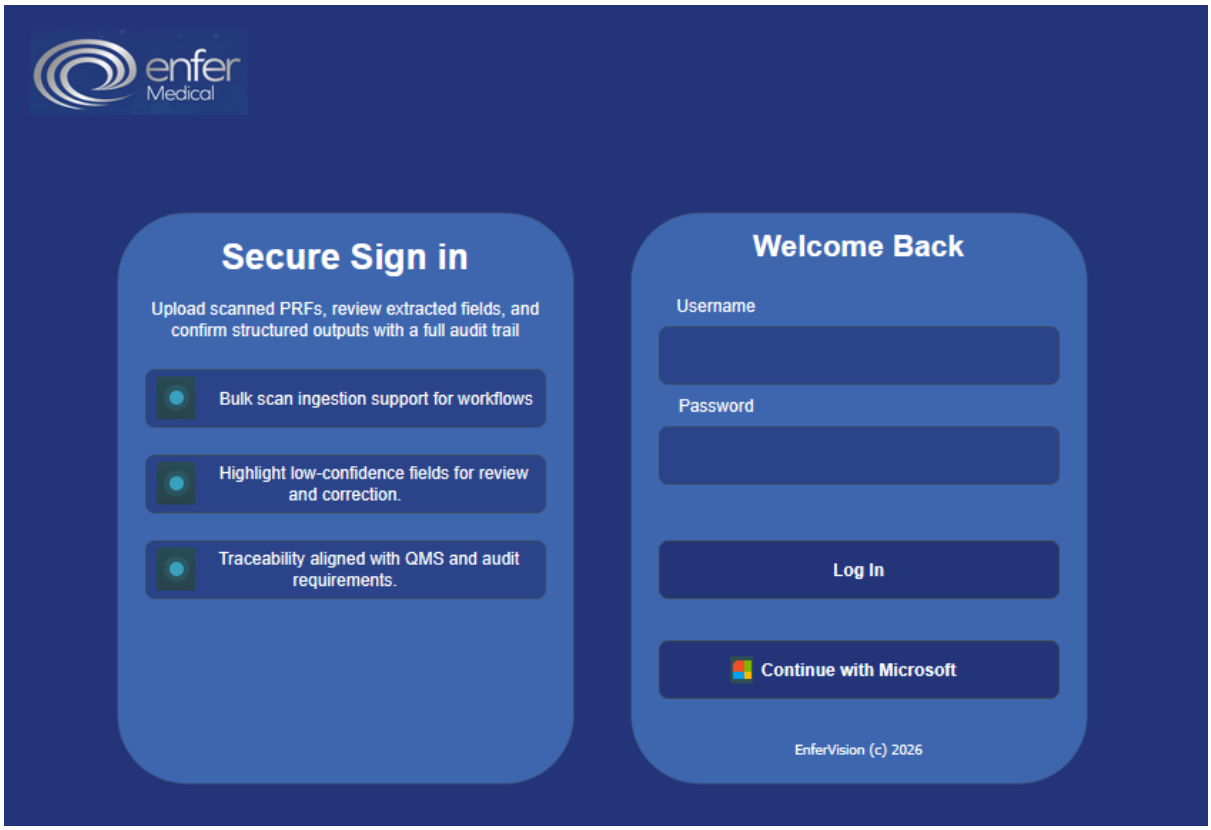


Figure 4: Mock-up layout for application login screen.

Upon successful authentication, users will be presented with a table list of PRFs. Figure 5 shows a mock-up of this table list. Users can select a specific record to open the review screen. A search field is also included to allow users to scan the unique PRF barcode to open the record review screen for the PRF. This will be the primary approach to accessing records as users will work in batches of up to 20 records and PRFs will be scanned and ingested ahead of users reviewing their batch.

On opening a record, the user will be presented with the scanned PRF image, accompanied by controls for zooming and rotating the document. Alongside the image, the data extracted from the PRF is displayed, with each field annotated with an associated confidence level to indicate the reliability of the data extracted. (see Figure 6). Future iterations may include an option to manually upload a PRF.

Figure 5: Mock-up layout for table list of outstanding PRFs. Admin button will only be visible for administrative users.

Figure 6: Mock-up layout for review screen for scanned PRFs.

Users assigned an administrative role will have the option access the admin review screen as shown in Figure 7. This screen will provide a tabular list of active users showing username and assigned roles. Administrative users will have the option to remove users using a 'DELETE USER' button. In addition to accessing the user list, administrative users will also have the option to access the audit history as shown in Figure 8. The audit history screen will provide a tabular list of actions carried out by a user. Administrative users can see a timestamp of the action, the PRF ID (where applicable), the user who performed the action, the action type and a brief description of the action.

Figure 7: Mock-up of Admin view.

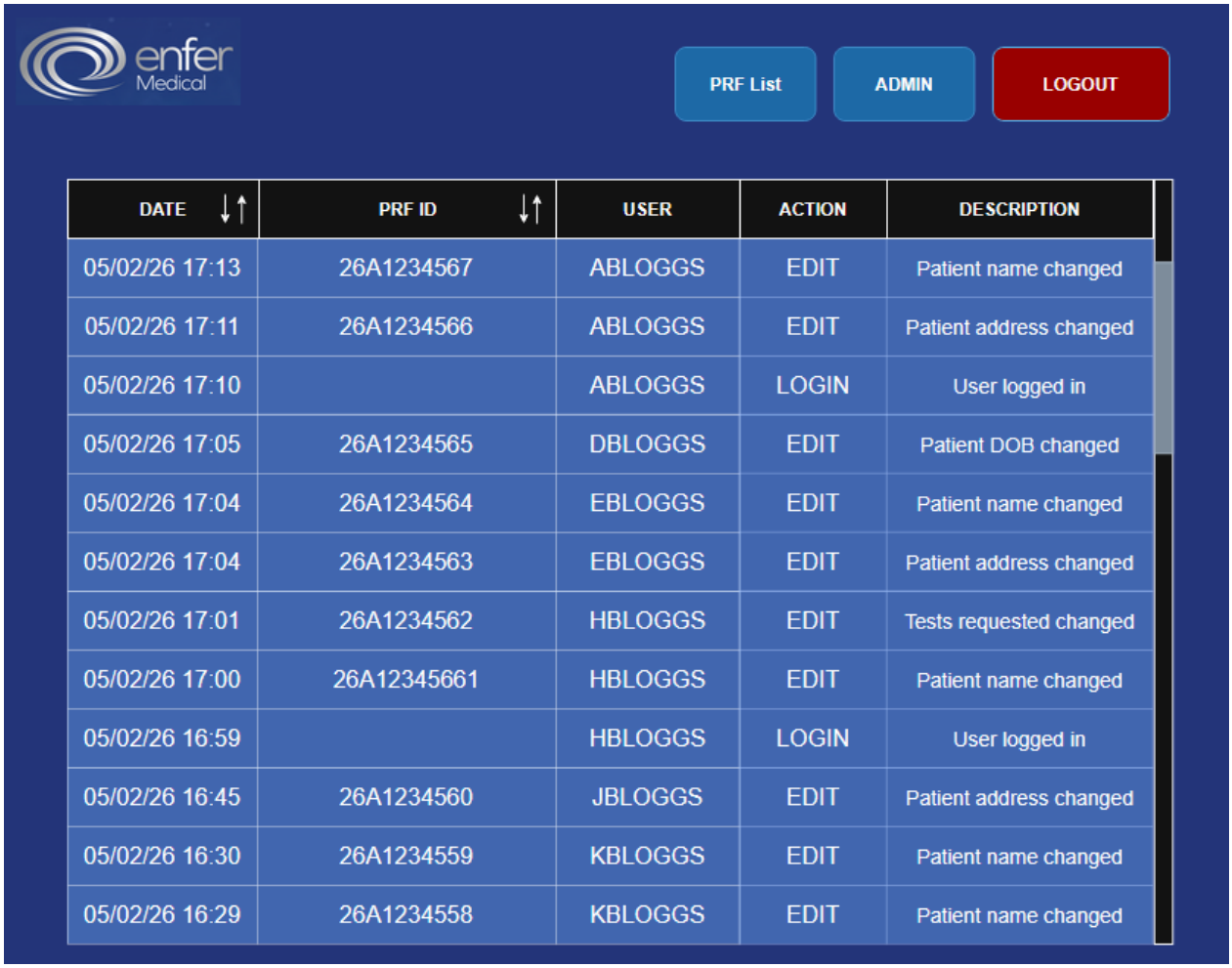


Figure 8: Mock-up of audit history review screen

3.3 Back-end

The back-end model is centred on the PRF as the primary entity, representing both the scanned document and its associated record, as shown in Figure 9. Each PRF progresses through a defined lifecycle that includes ingestion, extraction, review and confirmation. The structure supports traceability between the PRF, its extracted data, and any subsequent actions performed within the application. To ensure accountability and compliance with QMS requirements, the model incorporates a dedicated audit history entity that records all system and user actions. Audit events are associated both with the relevant PRF and the user responsible for the action. An export entity supports downstream integration, capturing when structured outputs are generated.

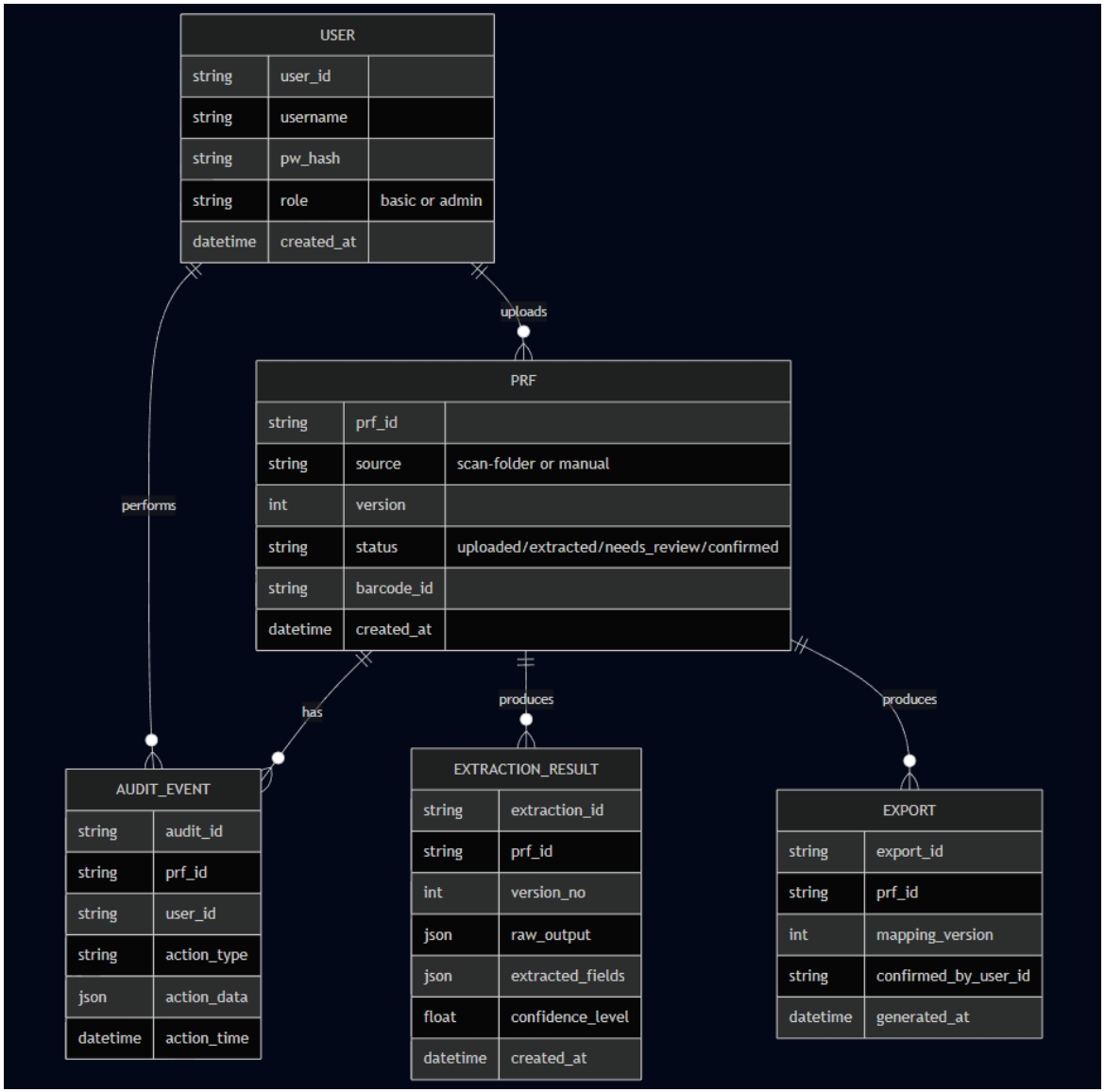


Figure 9: Entity relationship diagram for application back-end.

4. Planning

4.1 Front-end

4.1.1 Authentication

The application will provide two authentication methods - a standard login (*/login*) and a Microsoft OAuth login (*/login/microsoft*). The Microsoft OAuth option allows non-regular users, such as supervisors, who will have access to additional privileged functionality in future iterations, to authenticate using their work-issued Microsoft accounts.

The standard login option is intended for other users, primarily medical laboratory aides (MLAs). Both options will provide access to core functionality such as viewing PRFs, reviewing and editing of the AI-extracted data, and confirming patient data prior to transmission to the LIMS. In addition, both standard and non-standard users will have access to manual logout functionality and sessions will automatically expire after a defined period of inactivity.

4.1.2 Core UI Features

The primary workflow will consist of a table displaying all scanned and uploaded PRFs (*/dashboard*). This table will provide a filterable and sortable list of PRFs along with their current processing status (e.g. review pending or complete). A search function will also be available, allowing users to scan a PRF's unique barcode to access the corresponding review screen (*/prfs/:id/review*). This review interface will consist of a document viewer displaying the scanned PRF alongside the fields which have been populated with the extracted data. The document viewer will support zoom, rotate and PRF download functionality. Fields that could not be automatically completed, will be clearly indicated with a confidence level. Mandatory fields will be strictly enforced, and no confirmation will be permitted until all mandatory fields have been completed.

On initiating confirmation, an order summary will be provided, allowing users to either proceed with final confirmation or cancel the confirmation in order to make any additionally required amendments. On final confirmation, all fields will be locked, the PRF status will be updated to complete, and an audit record will be created showing all actions and the confirming user's initials alongside a timestamp of the final confirmation.

4.1.3 Access Control

In addition to the standard and non-standard roles described above, which provide access to core functionality, an administrative role will also be available. Administrative users will have access to all core functionality as well as access to a user administrative interface allowing for user management (*/admin/users*). This interface will allow administrative users to disable or delete user accounts as necessary.

Administrative users will also have access to audit log functionality (*/admin/audit*), allowing them to review and inspect user activity across the application. Access to administrative routes will be restricted and any attempts by standard or non-standard users to navigate to these routes will be strictly prohibited.

4.1.4 User Interface Testing

User interface (UI) testing will be completed using Cypress e2e to test functionality from a user's perspective. This testing will cover the following:

1) Standard and OAuth login
2) PRF data review, editing and confirmation
3) Disabling/deletion of users by an administrative user

In addition to Cypress e2e testing, user acceptance testing will be completed by lab staff in order to ascertain feedback for further development prior to production.

4.2 Back-end

4.2.1 Authentication

Secure user authentication will be provided using local username/password credentials in addition to Microsoft OAuth. Password hashing and salting will be employed for local authentication to ensure credentials are stored securely. On successful authentication users will be issued a JSON web token (JWT) to maintain authenticated sessions across API requests. Role-based access control (RBAC) will be enforced at the API level so that users can only access endpoints allowed by their assigned role (standard/non-standard or administrative). Any attempt to access a restricted endpoint will result in an appropriate error.

4.2.2 Upload and Ingestion

Automated PRF ingestion will occur through a scan folder watching service. This involves a PowerShell watcher script continuously monitoring a configured directory where scanned documents are deposited. Once a file is detected, it is moved to a processing folder, creating a PRF record and triggering the data extraction process. Duplicate documents will be prevented, and all ingestion events will be recorded in the audit log.

4.2.3 AI Data Extraction

Uploaded and ingested PRFs will be processed using mPLUG-DocOwl2 to extract the relevant structured data from the PRFs. For the handwritten elements that cannot be reliably extracted, an additional API call to Google Gemini will be implemented. Only cropped regions containing no personably identifiable information will be submitted, ensuring compliance with current company data security policies.

4.2.4 Confirmation and Output

Upon review and final confirmation of the extracted data by a user, further modification of the record is prevented. The confirmed data will be transformed into a structured XML format with the username of the confirming user being injected into the XML payload alongside a timestamp in order to provide traceability. The resulting XML payload will then be exported for downstream ingestion.

4.2.5 Auditing Functionality

An audit trail capturing all significant user and system actions will be maintained by the back-end. Auditable events will include PRF ingestion, extraction completion, data amendments by users and final confirmation of data by users. Each audit record will include the relevant user, a timestamp, the affected PRF (if applicable) and a brief description of the occurring event. Administrative users will have the capability to retrieve and export audit histories for review in order support quality assurance audits and non-conformance investigations.

4.2.6 Persistence Features

All core primary application data will be persisted in a locally hosted MongoDB database. Such data includes user accounts, PRF scans, extracted data, confirmed data and audit histories. This persistent storage provides continuity across sessions and allows for historical reviews. Scheduled data cleardowns and/or archiving routines will also be implemented to manage storage and comply with company data retention policies. These routines will only remove temporary and/or records exceeding designated cut-off times without impacting active or in-progress records.

4.2.7 Back-end Testing

The back-end will be supported by automated unit tests to validate key components such as authentication logic, RBAC enforcement, ingestion workflow, XML formatting and audit record creation. These unit tests will ensure core back-end functionality and ensure updates occur without regression.

4.2.8 Action Tracking

All implementation tasks will be tracked using Monday.com (see Figure 10). Task priorities will be set as either Low, Medium, High or Critical and grouped based on a status of Outstanding, In progress or Complete.

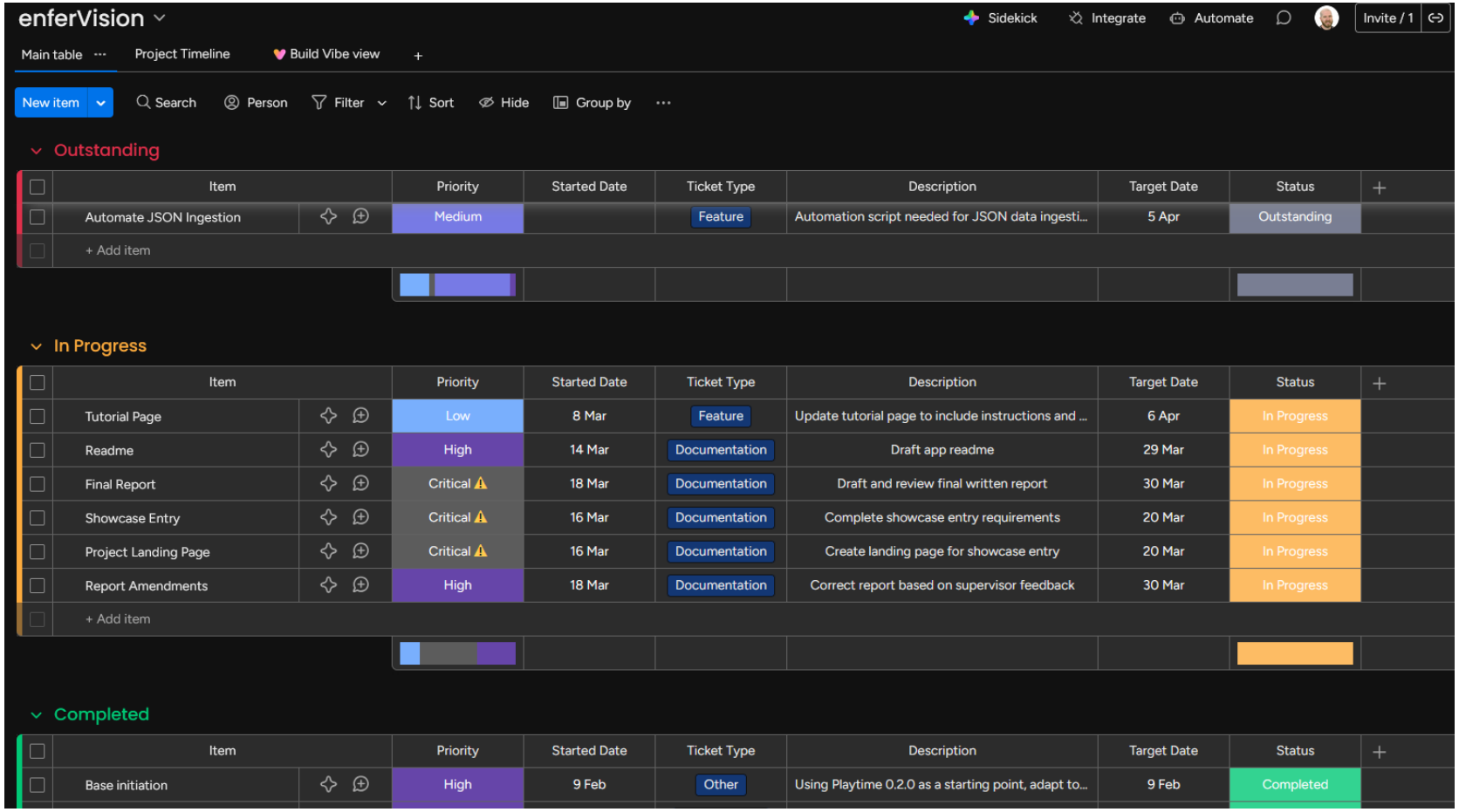


Figure 10: Monday.com Board for enferVision.

5. Implementation

5.1 System Model

enferVision is a full-stack application that incorporates PRF document ingestion, AI-based demographic data extraction and data management within a clinical workflow process. The enferVision system consists of the following 4 main elements:

1) Frontend – Handlebars UI
2) Backend – Hapi.js API
3) Database – MongoDB
4) AI Extraction Pipeline

The UI was implemented using Handlebar templates to provide a server-rendered web application that enables users to view ingested PRF scans, review and edit extracted patient demographic data details, add observations and confirm finalised details. Moreover, the UI supports both basic and administrative users, with administrative users having access to audit history and user account pages. The backend was built using the Hapi.js framework and functions as the main controller of the application. It handles HTTP requests and routing, as well as data entry validation using Joi schemas and manages data transfer between the database, the UI and the AI-based service. In addition, it also manages user authentication and session handling, as well as audit event logging.

MongoDB was used as the primary data store, which allows for flexible document-based storage as previously discussed. The MongoDB store holds user and authentication data, PRF records, extracted PRF details and their related confidence levels, observations and audit history logs. Hardware and time limitations prevented the application of mPLUG-DocOwl2 and other document processing services. These approaches require significant computational resources which are currently unavailable, as well as extensive configuration. In lieu of a document processing service, Tesseract was initially used to apply OCR-based extraction, but this was replaced with PaddleOCR due to limitations where label skewing occurred. However, due to extensive parsing requirements, PaddleOCR was then replaced with a full OpenAI API call for PRF data extraction.

The entire scanned PRF document is sent via the API call to OpenAI where structured data is extracted using a defined schema and specific rules. Confidence levels are also provided with each field, and results are output to JSON files for ingestion. At a high level, as shown in Figure 11, the system involves PRFs being pre-labelled with unique barcodes. The PRFs are then bulk-scanned with each PRF record created and named using the unique barcode. A Python script, which currently needs to be manually executed, executes the API call for each scanned PRF, and a structured JSON file is created containing the AI-extracted details for the PRF. Node.js processing scripts, which are also currently only manually executable, ingest the PRF records and the JSON files.

A user logs in and finds a PRF by using a handheld scanner to enter the barcode into the PRF table search field. Within the PRF details page, a user can then compare the extracted data to the scanned PRF image. Confidence levels next to each field indicate extraction accuracy and when the user is satisfied the extracted data is correct, they can use the confirm button to lock the record and create an XML file output containing the extracted data for downstream ingestion.

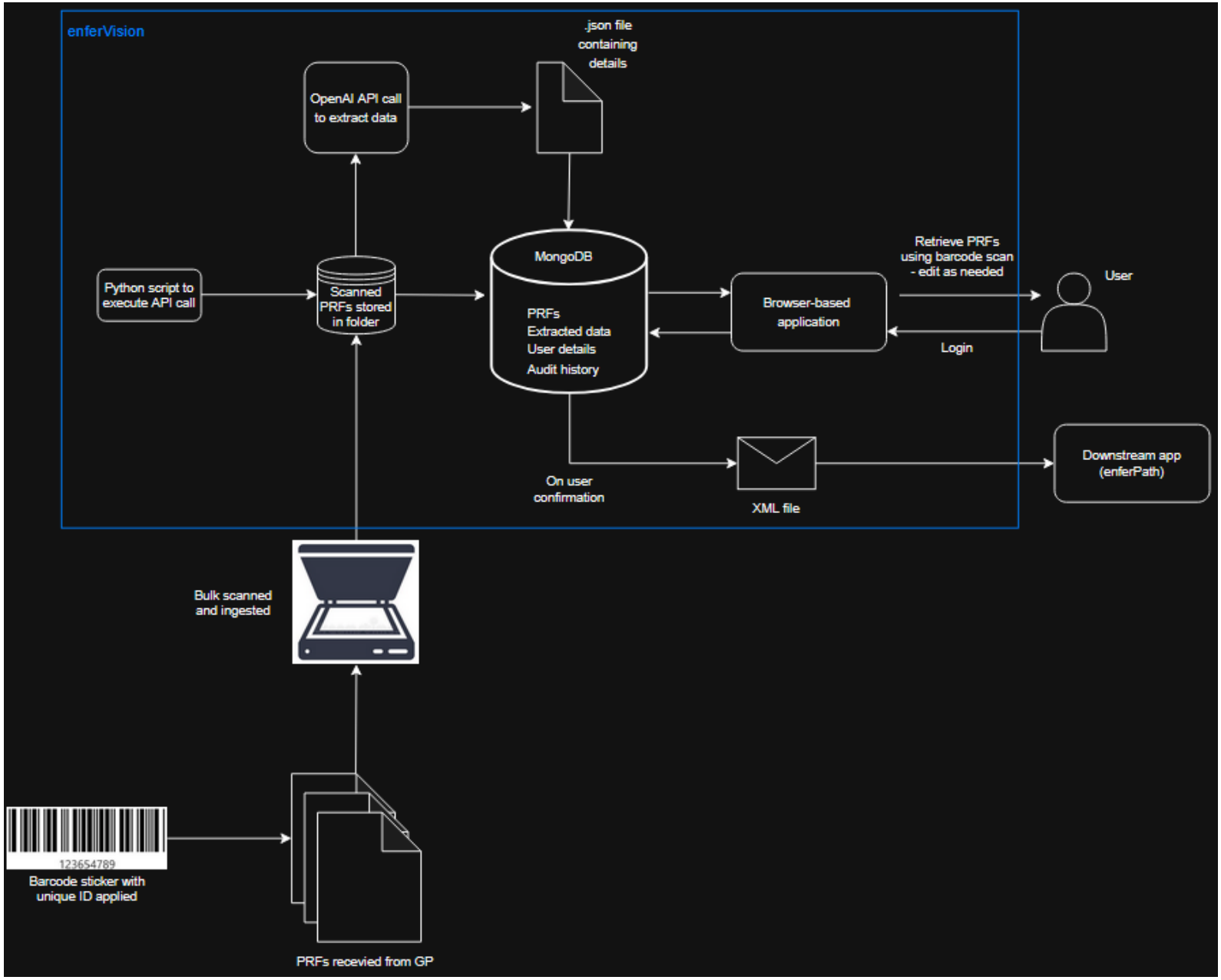


Figure 11: Architectural Diagram of Current Application Workflow

5.2 Feature Summary

5.2.1 Authentication

RBAC is applied to control access to application functionalities. Users must log in to access the application, user accounts can only be created by an administrative user and account passwords undergo salting and hashing. Function permissions are determined by role, standard or administrative. Standard users can view PRFs, add or edit details, submit observations and confirm records as complete and accurate. Administrative users have the same privileges as standard users but can also create or delete user accounts and access audit history logs. Session-based authentication provides secure access throughout the app lifecycle.

5.2.1 PRF Management

PRF records are created by a bulk-scanner and its associated software which are currently manually ingested using a Node.js processing script. A user can also create PRF records by adding the PRF barcode to the PRF table and if a scanned file with a matching name exists in the scans folder, the two will be linked. This acts as a workaround where, in rare circumstances, a scan fails and manual intervention is needed. Future iterations may remove this function and replace it a with a versioning step, however, the bulk-scanning process is not currently capable of versioning scanned records. Duplicate PRFs are not permitted within the app.

5.2.2 Detail Entry and Validation

Each PRF record acts as a container for the associated patient demographic data and user observations. Patient details can be entered manually or populated using an AI service API call. Users may also edit details entered via the API call. All data field inputs are validated using Joi schemas in order to ensure correct data formats, such as date and time patterns, valid categorical data values, such as for gender and fasting, and required fields are completed before the record can be confirmed. Any validation errors are presented to the user on the details page. The scanned PRF is embedded using *iframe*. Thus, the browser's native PDF viewer renders the file and provides numerous functions such as zoom and rotation to assist data field reviewing.

5.2.3 AI Data Extraction

An AI-based extraction pipeline is integrated into the system which is used to automatically parse PRF scans and identify the required demographic data. Scans are processed via an API call to OpenAI where the required data fields are extracted according to a predefined schema.

Confidence scores are provided for each extracted field value, and a JSON file is created for ingestion into the application. This approach provides greater accuracy than OCR-based approaches, has no significant hardware requirements – a particular issue encountered with locally-hosted document processing services, provides greater scalability, and requires significantly less post-extraction processing to handle noise extracted alongside the required data.

5.2.4 Observations

Users can add free-text observations to PRF detail records which are displayed alongside the PRF details with the UI. This provides users the option to record comments and issues and allows them to provide reasons for certain decisions related to the record and detail changes. It also allows for input from multiple users where a problematic record is worked on across multiple shifts. These observations are timestamped and linked to the user to allow a thread to be easily followed. This is particularly important if a thread needs to be compared against an audit history, such as during a non-conformance investigation.

5.2.5 PRF Locking and XML Generation

Once all the necessary details have been completed and a user is satisfied that the record is as complete as possible, a PRF can be confirmed for downstream ingestion. Confirmation triggers the generation of a structured XML output file as well as locking of PRF records in order to prevent further modification and generation of successive XML files. The record is preserved in its finalised state, ensuring data integrity and accurate traceability.

5.2.6 Audit Logging

The system maintains a comprehensive audit trail of specific actions in order to support traceability and investigations as well as key performance indicator reports. User authentication events are recorded in order to track what users are logged in and for how long. PRF creation is recorded to track manual and automated events and confirmation of completed PRF records is also recorded to provide record processing timelines in addition to standard traceability. Furthermore, detail additions and updates are also recorded to ensure user accountability. Lastly, user account creation and deletion is recorded as evidence for compliance with onboarding and offboarding policies. The audit history logs include user identity, timestamps, and before and after states (where applicable) and are a critical element for supporting clinical traceability and meeting compliance standards.

5.3 UI Implementation

5.3.1 Login Page

The enferVision application opens with a login interface where users authenticate using their email and password. As shown in Figure 12, the login interface is designed to be simple and user-friendly with validation ensuring correct data input before authentication proceeds.

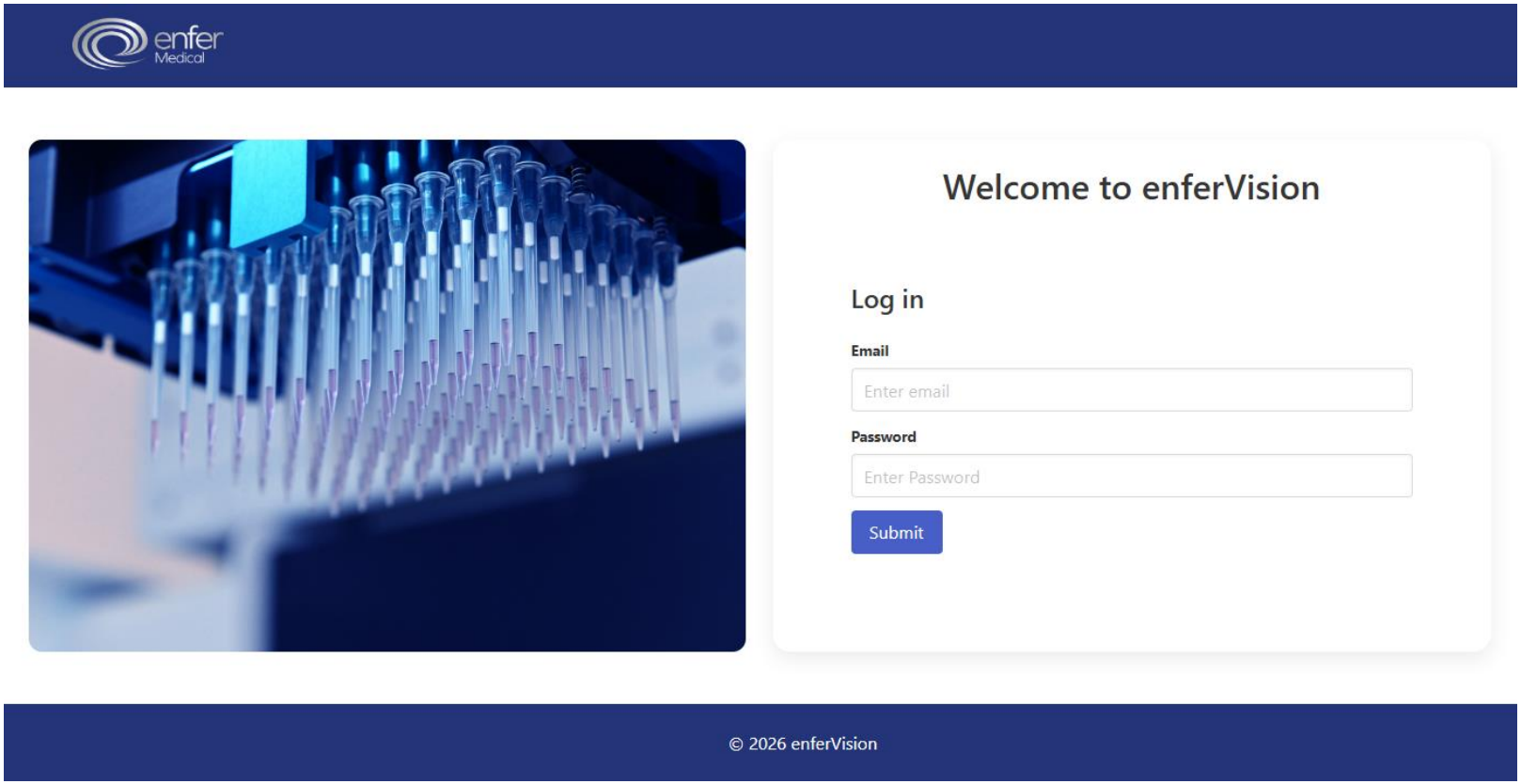


Figure 12: Login Page

5.3.2 Dashboard

On successful login, users are redirected to the dashboard (see Figure 13). This page displays existing PRF records along with their creation date and status and it provides the option to create a new PRF record. From the dashboard, users can open PRF records to view details or navigate to the tutorial page, and administrative users can access the admin and audit history sections. The table is currently not sortable and requires additional filtering options (e.g. by date) to improve user experience.

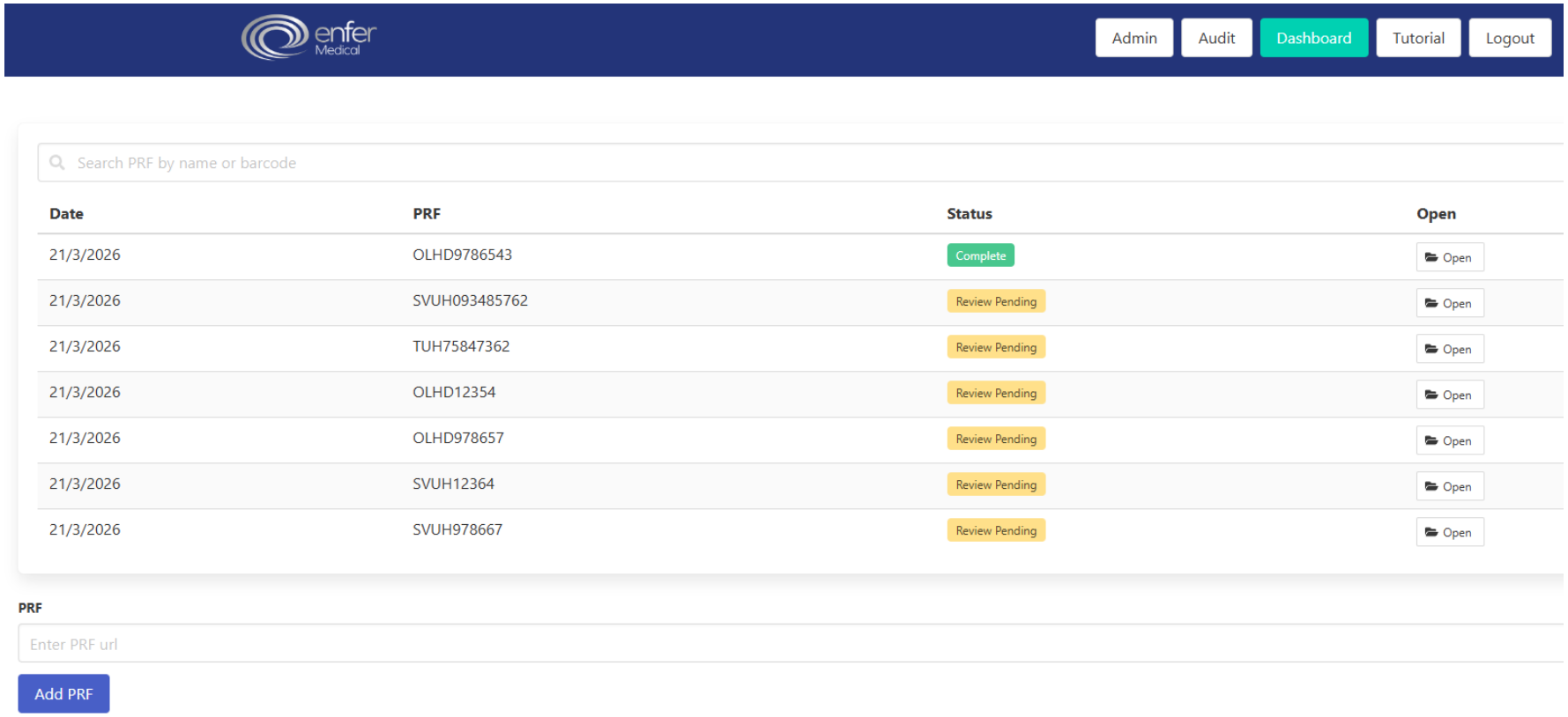


Figure 13: Table of Ingested PRFs

5.3.3 PRF Details Page

Opening a PRF record brings the user to the PRF details page (see Figure 14) where the user can see the scanned document alongside the relevant data entry fields. Users can see the data input from the API call with a traffic light-based confidence scoring system to guide reviewing. Users can enter data into missing fields or update existing entries. Validation ensures required fields are completed and meet format requirements.

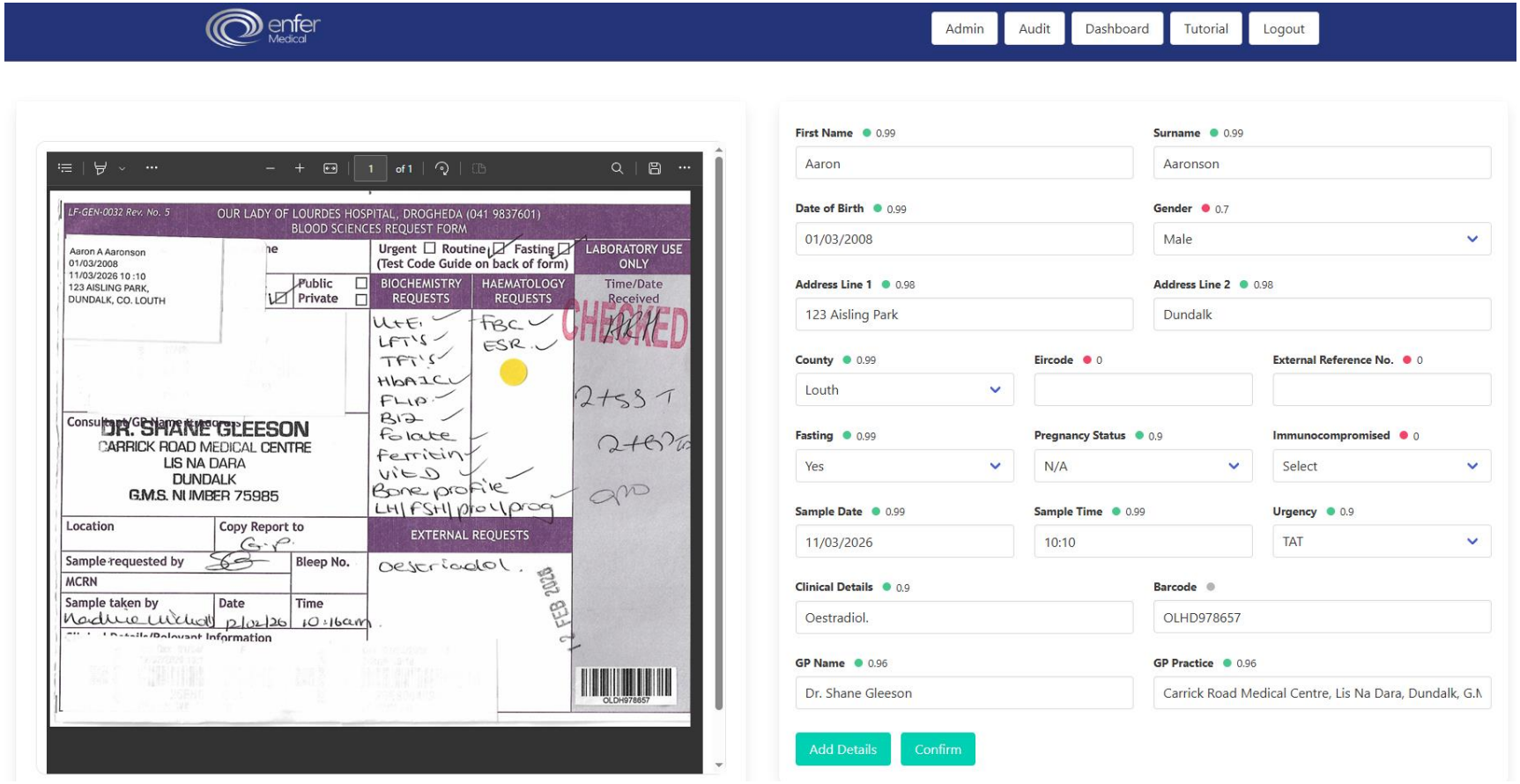


Figure 14: Details Page

5.3.4 Locked PRF View

As shown in Figure 15, once a PRF is confirmed, it is locked and can no longer be edited by a user. Locked records are clearly indicated with a banner and removal of the confirm button.

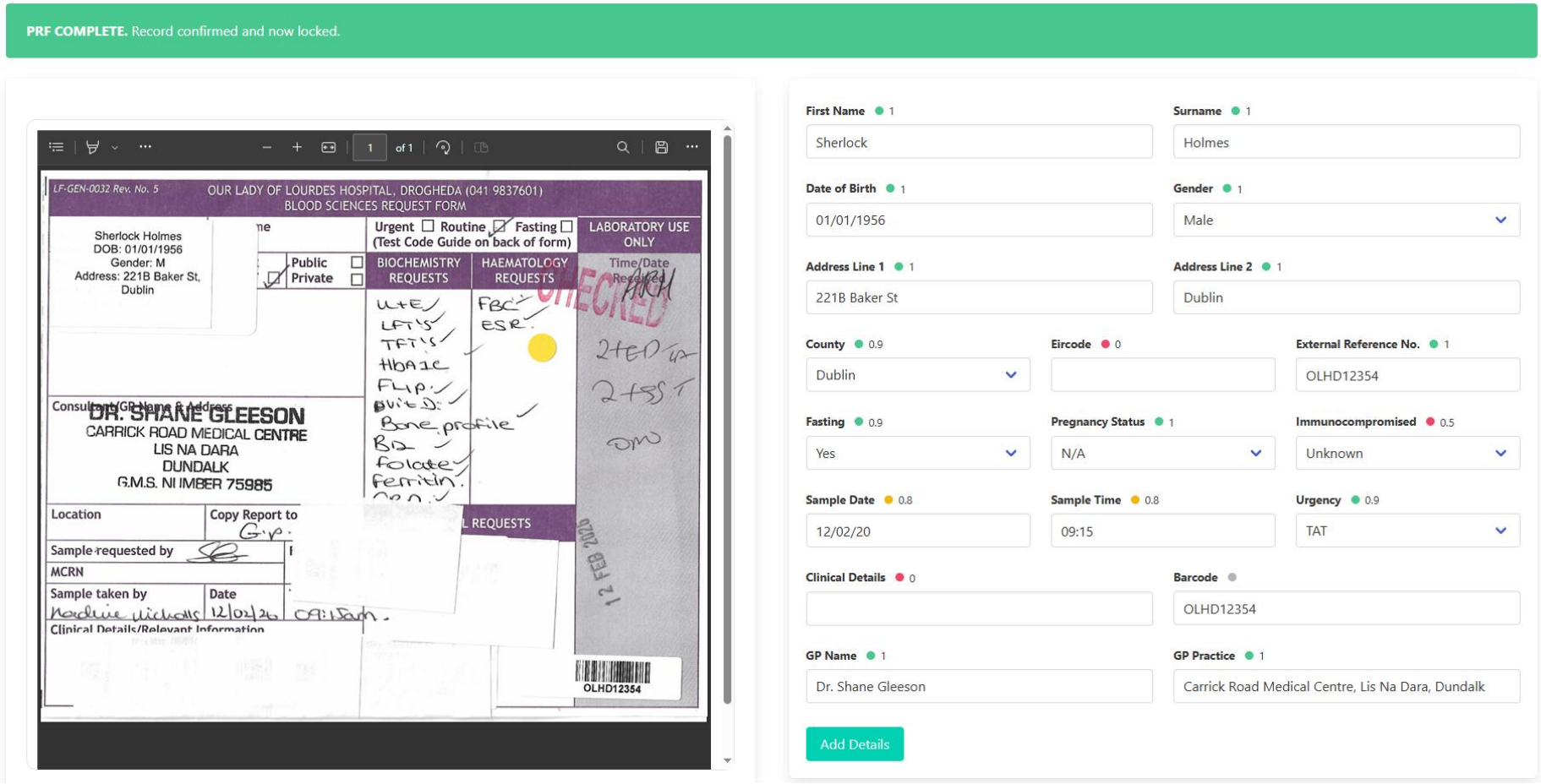


Figure 15: Locked Details Page

5.3.5 Observations Section

Beneath the scanned document and data entry fields on the PRF details page, an observations section allows users to add comments against a PRF record (see Figure 16). Observations are displayed in chronological order with usernames and timestamps to allow traceability and collaboration where multiple users/shifts work on a record with an issue.

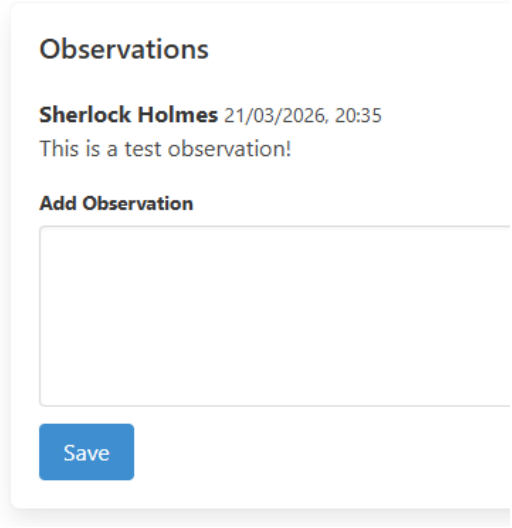


Figure 16: Observations Section of Details Page

5.3.6 Audit History – Authentication

The audit history interface includes a dedicated tab for viewing user login and logout activity (see Figure 17). This allows administrative users visibility on user access patterns, ensures accountability and provides traceability.

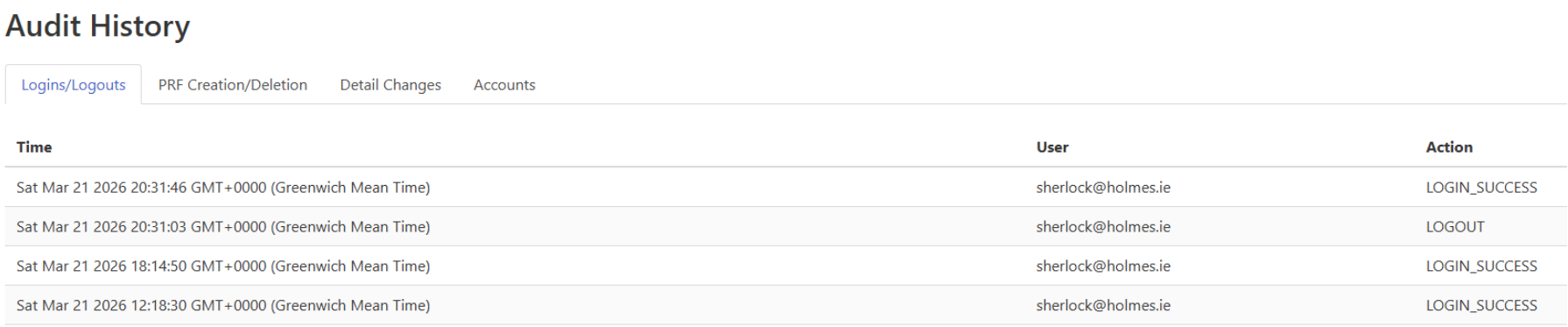


Figure 17: Login/Logout Audit History

5.3.7 Audit History – PRF Creation

The audit history interface also includes a dedicated tab for viewing PRF creation (see Figure 18). This allows administrative users visibility on PRF creation events with usernames and timestamps to ensure accountability and traceability.

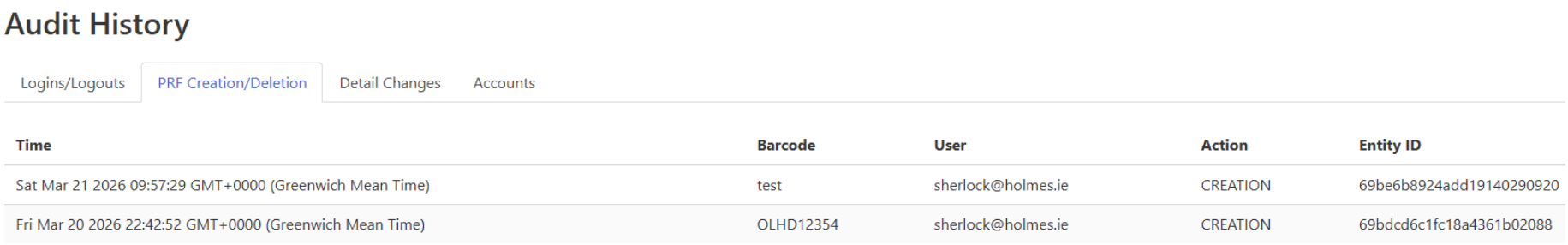


Figure 18: PRF Creation Audit History

5.3.8 Audit History – Detail Changes

The audit history interface also includes a dedicated tab for viewing PRF detail changes and confirmations (see Figure 19). This allows administrative users visibility on detail additions and update events with usernames and timestamps to ensure accountability and traceability.

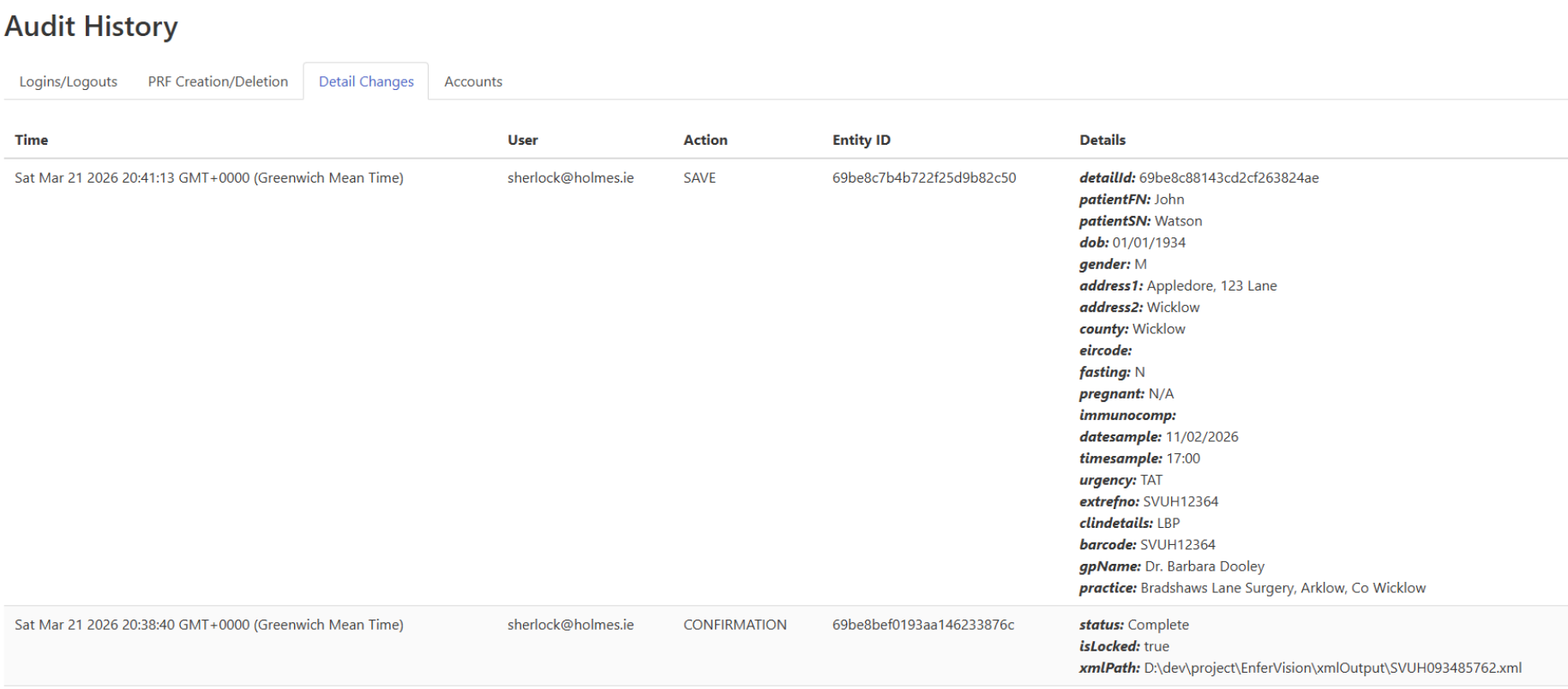


Figure 19: Detail Changes Audit History

5.3.9 Admin – User Management Table

Administrative users have access to a user management table on the admin page (see Figure 20). The table provides a list of all accounts and associated details including first name, last name, email and role. User accounts can be added or removed by an administrative user, but administrative users cannot remove their own account.

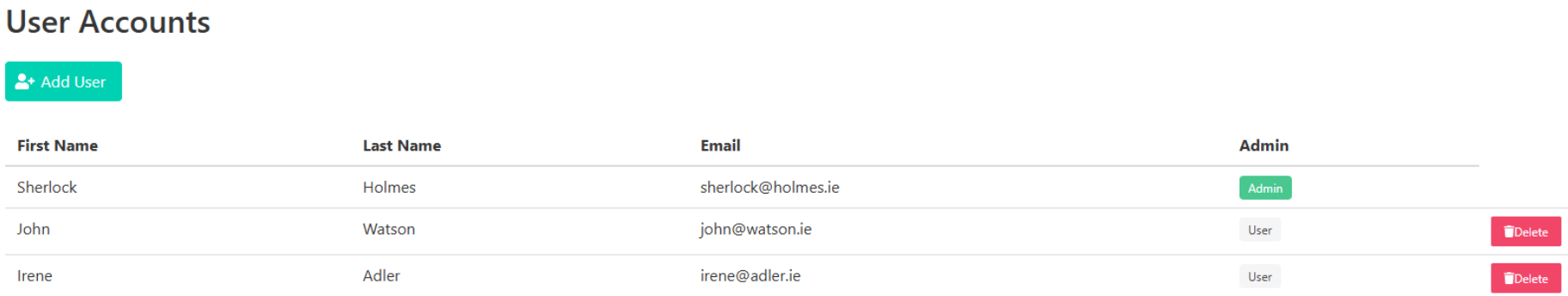


Figure 20: Admin Route User Accounts Table

5.3.10 Admin – Add User Page

Administrative users have access to a user creation interface where user accounts can be created as part of the onboarding process (see Figure 21). Accounts can be created as standard or administrative.

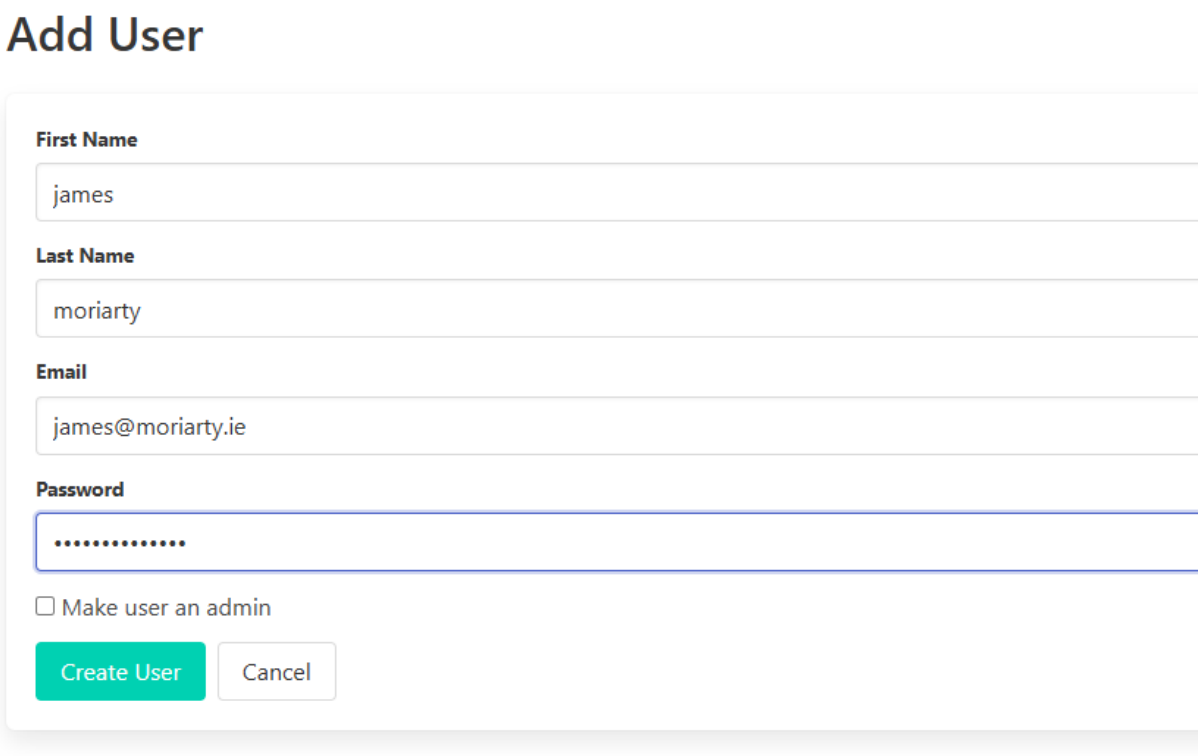


Figure 21: Admin Route Add User Page

5.3.11 Audit History – Account Events

The audit history interface includes a dedicated tab for viewing account creation and deletion events (see Figure 22). This allows traceability of administrative actions to ensure accountability and traceability.

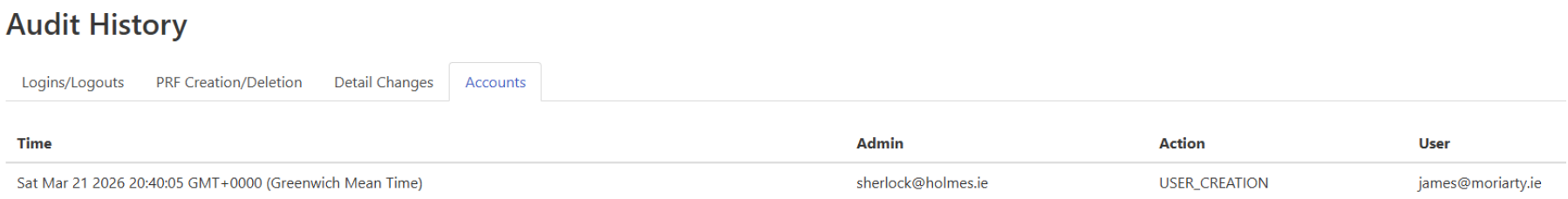


Figure 22: Accounts Creation/Deletion Audit History

5.4 Back End API

5.4.1 Routing Structure

The enferVision application maps HTTP requests to specific controller functions. These routes are defined centrally (within web-routes.js) and grouped by functionality. The current route groups include:

1) Authentication routes used for login, logout and session establishment.
2) Dashboard routes to provide PRF table access and PRF creation.
3) PRF routes to provide PRF viewing, detail entry, confirmation and observations.
4) Admin routes to support user account management and audit history viewing.
5) A static asset route to serve public files such as images, scans and styling resources.
6) A tutorial route to provide tutorial page access.

5.4.2 Controllers

The application controllers function as the primary interface between system operations and incoming requests. They coordinate validation, sanitisation and persistence in conjunction with Joi schemas, utilities and MongoDB store modules before data is committed. Each controller is responsible for a specific functional area:

1) The accounts controller handles user authentication, session management and login validation.
2) The dashboard controller manages dashboard operation which includes displaying PRFs and creating new PRF records.
3) The PRF controller manages PRF record viewing, the addition and updating of details, PRF confirmation, XML generation and the addition of observations.
4) The admin controller supports administrative functions including user account overview, the creation and deletion of users and audit history viewing.
5) The tutorial controller allows access to the tutorial page which is currently a work-in-progress.

5.4.3 Validation

Incoming data for key operations is validated using Joi schemas to ensure data integrity and consistency. This ensures required fields are present where necessary, data types and formats are as expected (e.g. date and time patterns) and categorical data values are enforced as expected downstream. Input data validation occurs at the route level within the Hapi framework with error handling applied to provide the user with useful feedback where data validation rules are not met.

5.4.4 Data Access Layer

The application implements a dedicated data access layer in order to manage MongoDB interaction (see Figure 23), and each entity is managed through a corresponding store, including the user, PRF, detail, observation and audit stores. These stores cover the required database operations, including create, read, update and delete (i.e. CRUD) functionality, and avoid the need for direct database queries within the controllers.

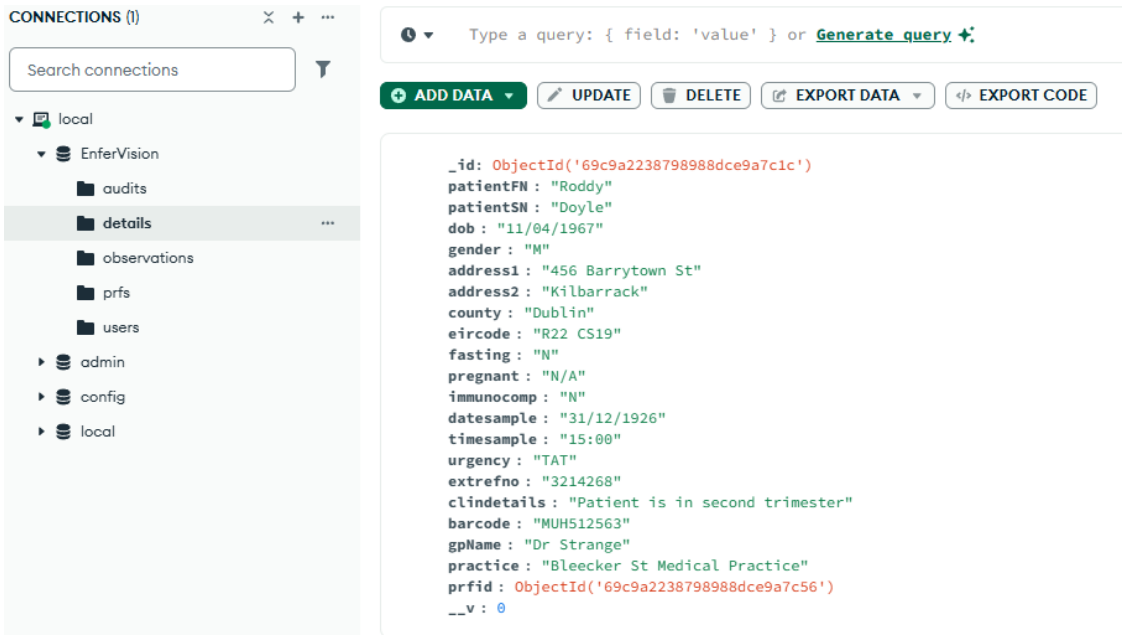


Figure 23: MongoDB screenshot showing collections and detail document.

5.4.5 Authentication Mechanisms

Two authentication mechanisms are implemented in order to support both web-based interaction and API access – JWT authentication (API layer) and cookie-based authentication (web layer). The JWT approach secures API endpoints with users authenticating through a dedicated API route which generates a signed token containing user identification data.

The token generated is included in subsequent requests and validated with each call. The JWT tokens are time-limited (currently 1-hour) and verified using a shared secret to provide secure API communication. Hapi's cookieAuth mechanism is used in the web interface to manage user sessions. After a successful user login, a session cookie is created and applied to authenticate subsequent requests. This provides secure access control across protected routes while maintaining a user-friendly user experience. The Hapi cookieAuth session-based approach is the default authentication mechanism for web routes and the JWT authentication approach is applied to API routes. This allows both browser-based usage as well as programmatic access via the API. User passwords are stored using bcrypt hashing and salting prior to storage (see Figure 24). During authentication, submitted credentials are compared against the stored hash using bcrypt's secure comparison function. Active session expiration was not achieved and remains to be implemented.

Figure 24: Screenshot of user document with hashed password.

5.5 Deployment Infrastructure

The current enferVision implementation is deployed in a local development environment and the application runs on a single endpoint which hosts both the back-end server and the database. The back-end is implemented using Node.js and the Hapi framework, with the application being executed via a Nodemon development server and MongoDB acting as the primary database. Configuration values, including authentication secrets (i.e. cookie name and password), the database connection string and the OpenAI API key are managed using environment variables stored in a .env file.

Local file storage is currently applied for PRF scans, AI-extracted data JSON files and generated XML outputs. While this simplifies development and testing, adaptation is required and under review for distributed network deployment. Future iterations will use a centrally hosted MongoDB instance on a company virtual machine which will allow multiple application instances to connect to the shared database.

In addition, network-accessible shared folders will be used for storing scans, JSON files, and XML outputs, improving accessibility, scalability, and reliability across the system. Containerisation using Docker will also be applied to standardise deployment across endpoints, ensuring consistent endpoint environment setups while simplifying scaling and maintenance.

5.6 Continuous Integration/Deployment

Automated unit testing, while not yet extensive, is performed using Mocha with tests covering API endpoints and data models. These tests are used to validate functionality following an update during development. Current unit tests are shown in Figure 25. Cypress e2e testing has also been implemented to test functionality from a user perspective (see Figure 26).

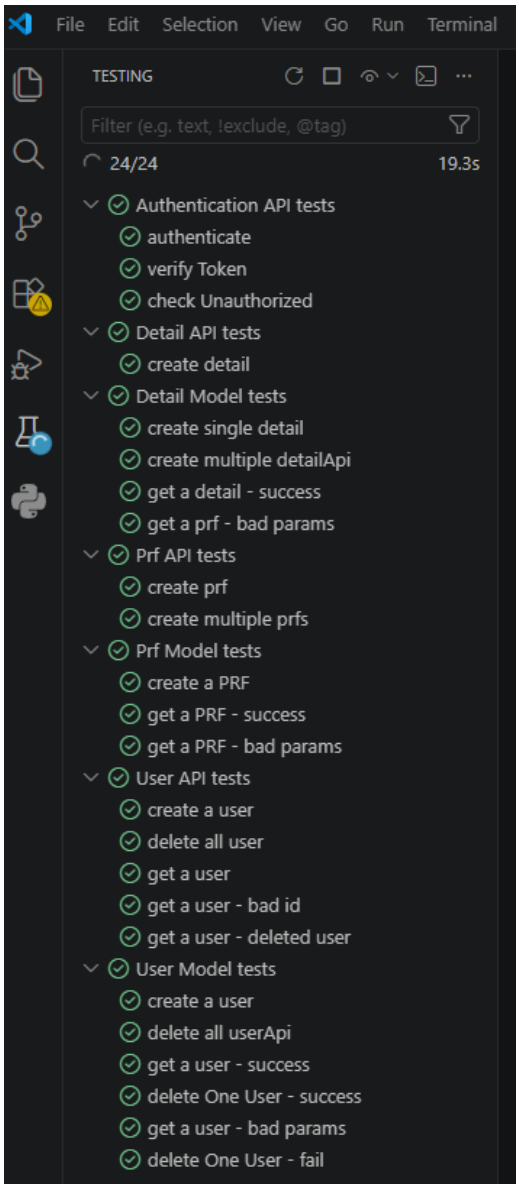


Figure 25: Unit tests currently configured for enferVision.

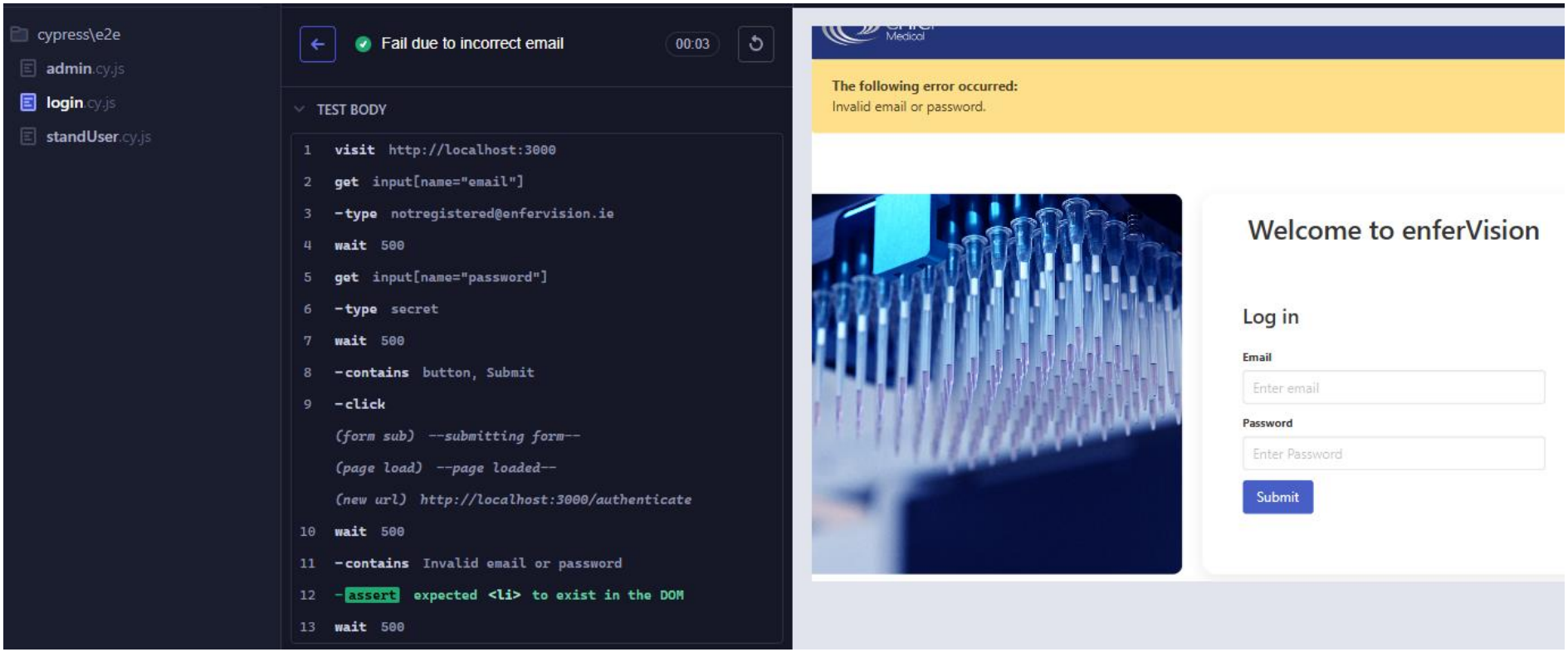


Figure 26: Cypress e2e testing UI.

The typical approach is to implement a feature and/or change, run an automated test and resolve any issues identified following the test. However, while the order of development and testing operations stated is not always strictly adhered to, this test-driven development approach is central to the process, and no features or changes are introduced unless sufficient testing and validation are carried out. Development and testing are further supported through the implementation of Swagger documentation for the API endpoints (see Figure 27). This provides a UI for testing API routes and allows request payload structures and parameters to be examined. Furthermore, as mentioned, the application is run locally using Nodemon, allowing quick feedback by automatically restarting the server when changes are made. Version control is managed using GitHub, allowing changes to be tracked and maintained throughout the development lifecycle.

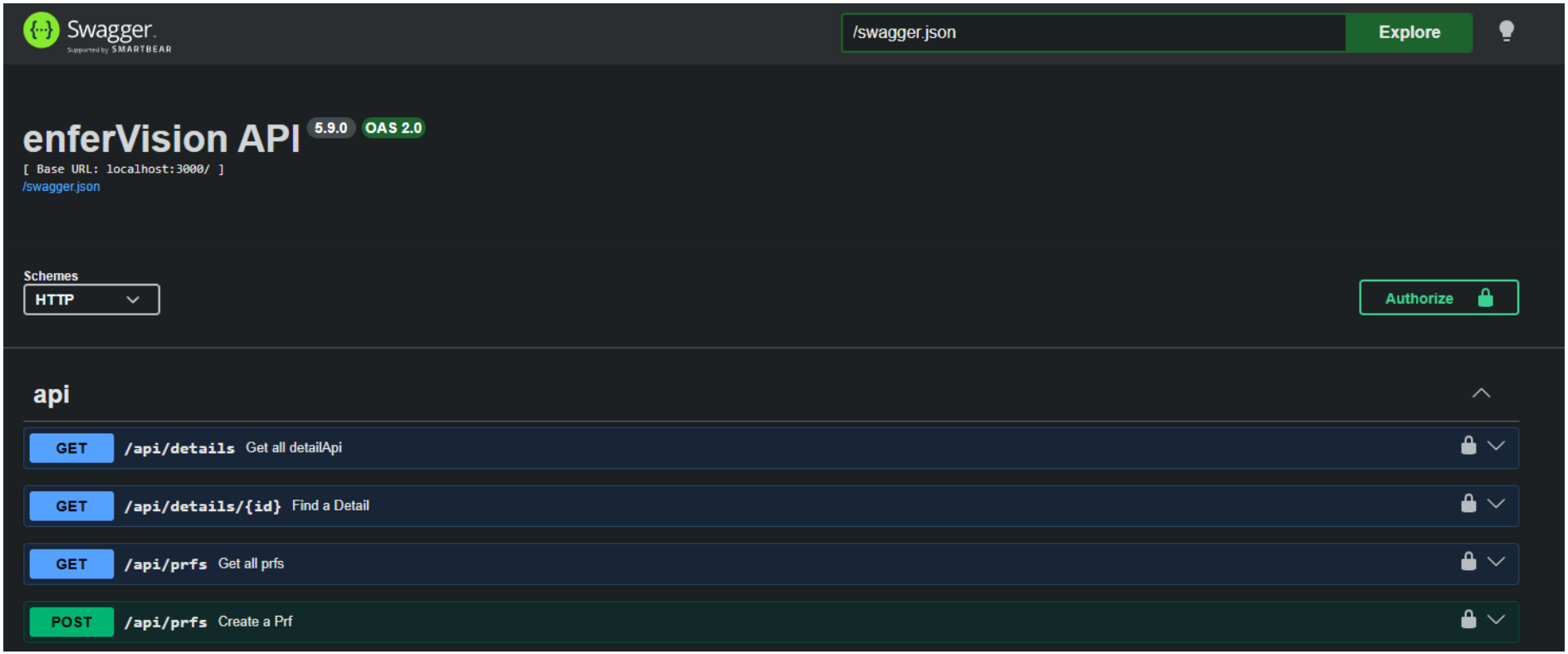


Figure 27: Swagger API Documentation.

6. Evaluation

6.1. Functional Evaluation

The enferVision application meets many of the key goals set out at the beginning of the project but with many areas requiring further development:

Key Goals

1) Build secure locally hosted browser-based application capable of ingesting GP PRFs in bulk and storing each as a unique record.
    - This goal was achieved in terms of application architecture and record storage but automation of PRF ingestion remains to be implemented.
2) Apply an AI-based document processing service to automatically extract the relevant details from each PRF.
    - This goal was achieved through the use of an external AI-based API call rather than through a locally-hosted document processing service as originally planned.
3) Build an interface that allows users to edit and confirm the extracted data.
    - This goal was achieved to a reasonable level with extracted data fields and confidence levels presented alongside the scanned PRF.
4) Add functionality to transform the confirmed data into an XML formatted file.
    - This goal was achieved, however the current XML generation process uses a custom build rather than an XML generation library which would improve flexibility and reduce maintenance.

Stretch Goals

1) Provide secure authentication and basic security measures (hashing, salting, sanitisation etc.).
    - This goal was achieved using JWT authentication for API access, cookie-based authentication for web sessions, password hashing and salting using bcrypt and input sanitisation.

2) Provide basic auditability and administrative oversight.
    - This goal was achieved with an audit history pipeline that logs key events. In addition, administrative users can manage user accounts and review audit histories.
3) Show reliability with unit tests and Cypress.
    - This goal was also achieved with unit tests as well as basic Cypress e2e testing. Testing needs to be expanded however, to cover more actions and events, and user acceptance testing is also yet to be implemented.

6.2. Performance

The performance of the system is generally acceptable; however no stress-testing has been performed. The AI-assisted extraction process performs efficiently, with data extraction taking approximately 5-10 seconds per PRF which is acceptable for the intended workflow. Live user acceptance testing using typical order volumes is still required however, to ensure performance meets requirements. The user interface is very responsive during required operations but again, further testing is required to ensure adequate responsiveness as currently all system aspects are locally hosted and, for example, moving the database to a remote server, may reduce responsiveness.

6.3. Accuracy

Accuracy is an important aspect of the system and a high-level of accuracy is required to ensure the system is deemed fit for purpose. The use of AI-based data extraction demonstrated a high-level of accuracy particularly when interpreting structured PRF fields. Interpretation of labels containing demographic data was very effective as was interpretation of various handwritten elements. It is important to note however, that handwriting interpretation has not been extensively tested and the accuracy level is merely observational based on confidence levels. A high number of varying PRFs along with appropriate statistical analysis is required to determine the true level of accuracy the system is capable of. It should also be noted that moving to a full API call for a PRF improved data extraction profoundly in comparison to using OCR-based extraction as the Tesseract and PaddleOCR approaches tested struggled with label skewing and variation, respectively (see Figure 28 for example labels).

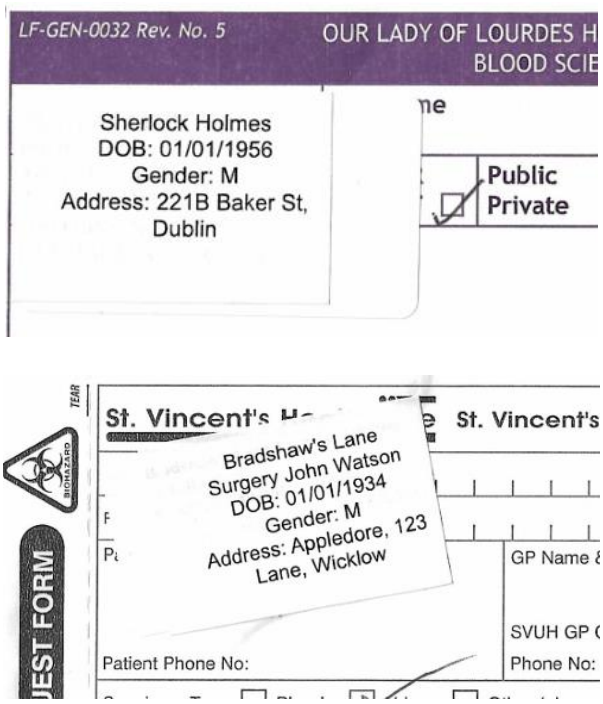


Figure 28: PRF label examples showing straight label (top) and skewed label (bottom).

Extraction noise was also a considerable issue. As shown in Figure 29, significant amounts of irrelevant text were extracted from the PRF images which required complex parsing. An OpenAI-written script was drafted for parsing and showed promising results but these results were dependent on the quality of the extraction and the parsing script itself would need regular updating based on label formats. This results in high levels of maintenance with increasing PRF variation while also slowing scalability. OCR-based extraction followed by parsing also made confidence scoring more complex as scoring would need to be based on both extraction and parsing.

Figure 29: Examples of extracted data, prior to parsing, from two different PRFs

6.4. Security and Privacy

Security and privacy considerations are critical when it comes to handling PRF data as they contain sensitive clinical patient information. Initially, a masking approach was taken, where the region of PRFs containing sensitive patient information was masked before carrying out an API call and the sensitive information was extracted using a locally hosted OCR-based process. This resulted in much lower accuracy however, increased numbers of empty fields, and significant issues when attempting to parse the extracted data into a meaningful payload.

The current implementation uses a full document API call and while this significantly reduced programming overhead and maintenance, improved scalability and accuracy, and simplified the implementation, it does raise additional privacy considerations which may contradict current company policies. From a GDPR compliance and data security compliance perspective, the use of enterprise-grade AI services such as OpenAI and Gemini for business are required to ensure data is not used for model training or other customer use. However, this still requires assessment in relation to data handling, storage and jurisdiction, and discussions are in progress with both external AI service account managers and external GDPR consultant vendors to examine justification and compliance. Customer service level agreements also require assessment to ensure the company is operating within required service obligations.

6.5. Limitations

Despite several successes in implementing the project, a number of limitations remain and further development is required.

- The main limitation is the reliance on an external API service for data extraction which, as previously discussed, comes with significant security and privacy concerns. Should the use of such services become a limiting factor, a locally hosted document processing service would need to be re-assessed, potentially introducing hardware costs as well as time and maintenance constraints. This external service reliance also has availability, connectivity and cost considerations. A locally hosted large language model (LLM) may also be considered.
- In addition, limited testing of the service and process has been carried out, and further assessment is required in order to ascertain its effectiveness in handling additional PRF variations, particularly those that are entirely handwritten.
- Another significant limitation is lack of automation for PRF ingestion, API calls and extracted data JSON file ingestion. These elements currently require manual execution and need to be automated, for example through file-watcher scripts or scheduled tasks, to support a scalable production environment.
- Another major limitation is that the project is currently locally hosted and the performance effects on moving to endpoint instances with a remotely hosted central database is unknown.

In summation, the system is not in a state for deployment and requires further development and additional considerations before live implementation (outlined in section 7.4 Future Considerations).

7. Reflection

7.1 Problems Encountered

The key issue encountered during development was the hardware limitation required to implement a locally hosted document processing service such as DocOwl and its variations due to extensive GPU requirements. Due to this limitation, as well as, pressing time constraints, it was decided to move to a more proof-of-concept model and implement a lightweight OCR-based approach. Tesseract was initially used and worked well for PRF demographic data labels but was quickly found to be ineffective where labels were skewed, even moderately. Tesseract was subsequently replaced with PaddleOCR which handled even skewed labels very effectively.

On examining the extracted output from a number of PRFs however, a considerable amount of variation occurred due to different label layouts and varying levels of noise extracted with the required data. In order to handle this variation and noise, highly sophisticated parsing would be required, making scalability difficult and process maintenance high. On reviewing these issues and the results from an OpenAI API call, it was decided to take the full API call approach discussed. The perceived accuracy and significantly lower implementation overhead of the full API-based approach resulted in a reconsideration of earlier design decisions and current company policies.

Numerous test failures were also encountered throughout development, and these were mainly linked to mismatches between Joi validation schemas and request payloads. These could have been avoided with more time dedicated to regular unit testing following updates. Moreover, database seeding proved very useful in early stages to provide users, PRFs and details for testing but made troubleshooting unit tests difficult due to expected record count mismatches. Seeding was subsequently removed. Lastly, unit testing was primarily carried using the command console which runs tests in the same sequential order each time. On running tests within Microsoft Visual Studio however, tests ran in different orders, resulting in failures. This was due to shared testing variables being altered in one test before being used in another. As a result, unexpected variables, such as IDs, were passed where they were not expected for a particular test. Isolation of testing and variables was applied to ensure test payloads were as expected and non-sequential testing was applied going forward.

7.2 Learnings

This project provided a significantly better understanding of various elements of full-stack development as well as the model-view-controller approach to development. A better understanding of the Hapi framework was developed as well as the advantages of a clear separation between routes, controllers and data access layers. This approach allows logic updates to be handled more independently without affecting the whole system. The effectiveness of using Joi validation schemas also became more apparent, particularly as enforcing date formats was greatly simplified in comparison to past projects.

A greater understanding of MongoDB was also achieved in using it as a data store, particularly in relation to the audit history data. The importance of unit testing was also reinforced as test failures highlighted unseen errors when testing at a UI level. Moreover, Cypress e2e testing highlighted several UI errors. Lastly, a significantly better understanding of OCR-based data extraction and AI-based data processing was attained, with both the strengths and limitations of each being particularly evident. In particular, an AI-based API call provides a very effective and efficient tool, offering considerable power.

7.4 Future Considerations and Outstanding Tasks

There are several items that were not achieved and further improvements that need to be considered.

Outstanding Items

- Current use of AI-based API call involves full scanned document. There are significant privacy issues here currently under review with external GDPR vendors, account managers and commercial team members.
- Automation of PRF ingestion, AI-based API calls and extracted data JSON file ingestion to enable scalability and provide an efficient workflow.
- Order summary GUI before final confirmation to allow a rapid final check. Necessity of this is under review.
- Scheduled data cleardowns and/or archiving routines to mange storage and comply with data retention policies.
- Additional filtering and sorting functions for PRF record table.
- Microsoft OAuth authentication for non-regular users such as sample receipt supervisors.

- Audit history search, filter and export functions to assist investigations and KPI reporting.
- User acceptance testing to attain user feedback in order to improve application and ensure fit for purpose

Future Improvements

- Record locking feature when a PRF is in the process of being edited would also be useful to reduce the chance of two users working on the same record, possibly where duplicate labelling has occurred.
- Developer role with enhanced functionality and Github OAuth authentication.
- Expanded unit and Cypress e2e testing to cover all actions.
- Additional error handling within code to ensure appropriate and secure feedback where errors occur.
- Containerisation to ensure standardisation across endpoints and simplify deployment.
- Penetration testing to ensure a secure application.
- Adaptation to create a version of the application for creating .csv files for uploading notifiable diseases to the Health Protection Surveillance Centre. This is already in the planning phase.
- Adaptation to create a version of the application isolated from the company network for logging samples where a breach or critical LIMS failure occurs. This is already in the planning phase as part of contingency planning.

8. References

AWS (2026) Amazon Textract. Available at: *https://aws.amazon.com/textract/* (Accessed: 31 January 2026).

AWS (2026) What's the Difference Between Block, Object, and File Storage? Available at: *https://aws.amazon.com/compare/the-difference-between-block-file-object-storage/* (Accessed: 30 January 2026).

AWS (2026) What's the Difference Between MongoDB and PostgreSQL? Available at: *https://aws.amazon.com/compare/the-difference-between-mongodb-and-postgresql/* (Accessed: 31 January 2026).

Bajrami, M., Zdravevski, E., Lameski, P. and Stojkoska, B. (2023) 'A Comprehensive Analysis of LayoutLM and Donut for Document Classification' , *The 20th International Conference on Informatics and Information Technologies*, CIIT 2023.

Beerten, T. (2025) OCR-Free Document Data Extraction with Transformers (2/2): Donut versus Pix2Struct on custom data. Available at: *https://medium.com/data-science/ocr-free-document-data-extraction-with-transformers-2-2-38ce26f41951* (Accessed: 31 January 2025).

Francis, S.A. and Sangeetha, M. (2025) 'A comparison study on optical character recognition models in mathematical equations and in any language', *Results in Control and Optimization*, 18, 100532.

GeeksforGeeks (2025) Difference between PostgreSQL and MongoDB. Available at: *https://www.geeksforgeeks.org/postgresql/difference-between-postgresql-and-mongodb/* (Accessed: 31 January 2026).

GeekforGeeks (2025) What is JSON? Available at: *https://www.geeksforgeeks.org/javascript/what-is-json/* (Accessed: 31 January 2026).

Gemini API (2026) Document understanding. Available at: *https://ai.google.dev/gemini-api/docs/document-processing* (Accessed: 31 January 2026).

Google Cloud (2026) Detect handwriting in images. Available at: *https://docs.cloud.google.com/vision/docs/handwriting* (Accessed: 31 January 2026).

Host IT Smart (2026) Difference Between Web Application and Desktop Application. Available at: *https://www.hostitsmart.com/blog/web-vs-desktop-application/* (Accessed: 31 January 2026).

HL7 International (2026) About HL7: What does HL7 do? Available at: *https://www.hl7.org/about/index.cfm?ref=nav* (Accessed: 31 January 2026).

HL7 Standards (2015) What is HL7? Available at: *https://hl7standards.com/what-is-hl7-and-why-does-healthcare-need-it/* (Accessed: 31 January 2026).

Hu, A., Xu, H. and Ye, J. et al. (2024a) 'mPLUG-DocOwl 1.5: Unified Structure Learning for OCR-free Document Understanding', *arXiv,* 2403.12895.

Hu, A, Xu, H. and Zhang, L et al., (2024b) 'mPLUG-DocOwl2: High-resolution Compressing for OCR-free Multi-page Document Understanding', *arXiv*, 2409.03420.

IBM (2026) What is file storage? Available at: https://www.ibm.com/think/topics/file-storage (Accessed: 30 January 2026).

InterfaceWare (2026) HL7 Message Structure. Available at: *https://www.interfaceware.com/hl7-message-structure* (Accessed 31 January 2026).

IONOS (2026) What is file storage? An explanation of the classic file system. Available at: *https://www.ionos.com/digitalguide/server/know-how/what-is-file-storage/* (Accessed 30 January 2026).

Ke, W., Zheng, Y., Li, Y., Xu, H., Nie, D., Wang, P. and He, Y. (2025) 'Large Language Models in Document Intelligence: A Comprehensive Survey, Recent Advances, Challenges, and Future Trends', *ACM Transactions on Information Systems*, 44(18), 1-64.

Kim, G., Hong, T. and Yim, M. et al., (2022) 'OCR-free Document Understanding Transformer', *arXiv*, 2111.15664.

LabManager (2024) 9 Steps on How to Perform a Laboratory Quality Audit. Available at: *https://www.labmanager.com/9-steps-on-how-to-perform-a-laboratory-quality-audit-20371* (Accessed 31 January 2026).

Lee, K., Joshi, M. and Turc, I. et al. (2023) 'Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding', *arXiv*, 2210.03347.

Mahadevkar, S.V., Patil, S., Kotecha, K., Way Soong, L. and Choudhury, T. (2024) 'Exploring AI-driven approaches for unstructured document analysis and future horizons', *Journal of Big Data*, 11(92), 1-54.

Makashyn, M. (2025) Traditional OCR vs AI-Powered OCR: Which to Choose for Your Business? Available at: *https://www.datavise.ai/blog/ocr-vs-ai-comparison* (Accessed 31 January 2026).

Mdn (2026) XML introduction. Available at: *https://developer.mozilla.org/en-US/docs/Web/XML/Guides/XML_introduction* (Accessed: 31 January 2026).

Medium (2024) Web vs Desktop Applications: Key Differences Explained. Available at: *https://medium.com/theymakedesign/web-app-vs-desktop-app-3841e8cb3996* (Accessed: 31 January 2026).

Microsoft (2026) Document Intelligence read model. Available at: *https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/ read?view =doc-intel-4.0.0* (Accessed: 31 January 2026).

MongoDB (2026) Comparing MongoDB vs PostgreSQL. Available at: *https://www.mongodb.com/resources/compare/mongodb-postgresql* (Accessed: 31 January 2026).

Murugan, R., Deivendran, P., Sony Kumari, D., Lokesh, B., Nimal, P. and Chandra Keerthy, S. (2025) 'AI-Powered OCR for Handwritten Documents with Low Quality and Degradation', *International Journal of Computer Scientific Technology & Electronics Engineering*, 1(2), 1-10.

Our Lady's Hospice & Care Services (2026) Medical referral forms. Available at: *https://olh.ie/medical-referral-forms/* (Accessed: 30 January 2026).

Ramotion (2026) Web Application vs. Desktop Application. Available at: *https://www.ramotion.com/blog/web-application-vs-desktop-application/* (Accessed: 30 January 2026).

Rhapsody (2026) HL7 Segments. Available at: *https://rhapsody.health/resources/hl7-segments/* (Accessed 31 January 2026).

Shaikh, A., Cochez, M., Diachkov, D., de Rijcke, M. and Yousefi. S. (2023) 'DONUT-hole: DONUT Sparsification by Harnessing Knowledge and Optimizing Learning Efficiency', *arXiv*, 2311.05778.

Sharma, A. (2023) Everything You Need To Know About OCR Technology And Its Applications. Available at: *https://medium.com/docsumo/everything-you-need-to-know-about-ocr-technology-and-its-applications-24a11e075814* (Accessed 31 January 2026).

SimplerQMS (2025) Laboratory Audits: Definition, Types, Requirements, and Process. Available at: *https://simplerqms.com/laboratory-audits/* (Accessed: 30 January 2026).

Singh, T.P., Gupta, S. and Garg, M. (2022) 'Machine learning: A review on supervised classification algorithms and their applications to optical character recognition in indic scripts', *ECS Transactions*, 107(1), 6233.

St James's Hospital (2026) GP and Hospital Referrals. Available at: *https://www.stjames.ie/healthcareprofessionals/referrals/* (Accessed: 30 January 2026).

Terzo (2024) Why Basic OCR Why Basic OCR Is Limited for Modern Contract Management Is Limited for Modern Contract Management. Available at: *https://terzo.ai/blog/why-basic-ocr-is-limited-for-modern-contract-management* (Accessed: 30 January 2026).

Torres, J.G.G (2025) Creating a document classifier and OCR: How to use Gemini 3 Flash to simplify the process. Available at: *https://medium.com/google-cloud/creating-a-document-classifier-and-ocr-how-to-use-gemini-3-flash-to-simplify-the-process-98029bea684a* (Accessed 31 January 2026).

W3 Schools (2026) What is JSON? Available at: *https://www.w3schools.com/whatis/whatis_json.asp* (Accessed 31 January 2026).

W3 Schools (2026) XML Tutorial. Available at: *https://www.w3schools.com/xml/* (Accessed 31 January 2026).

Ye, J., Hu, A. and Xu, H. et al. (2023) 'mPLUG-DocOwl : Modularized Multimodal Large Language Model for Document Understanding', *arXiv*, 2307.02499.

Appendix I – Ethics Questionnaire

This Ethics Checklist must be completed for all final year undergraduate, taught postgraduate and research projects in the School of Science and Computing.

View your response(s)

Respondent: **Daniel Cullen** (Group: CM-HDIPCS) Submitted on: Saturday, 29 November 2025, 11:19 AM

Ethics Checklist for Undergraduate, Taught Postgraduate and Research Projects in the School of Science and Computing

All students in the School of Science and Computing who are either (1) in the final year of an undergraduate/BSc degree, or (2) on a taught postgraduate/MSc programme **must complete this Ethics Checklist before conducting their project** regardless of the project type or discipline. The Checklist should also be completed by anyone (whether staff member or student) conducting a **research project** (whether programmatic or not) within the School.

The purpose of this Ethics Checklist is to **identify projects that will require formal ethical approval** from the School Research Ethics Committee, or the SETU Research Ethics Committee, before they can proceed.

Students/applicants should note that this Ethics Checklist is a **formal declaration**, and great care must be taken to **answer all questions accurately**. Students should consult with their project supervisors/advisors regarding any aspects or questions that they are unsure of before completing and submitting the Ethics Checklist.

Students/applicants must **answer all questions** presented to them until the Checklist questionnaire is completed.

Feedback Report

No human experimentation issues (UG).

No animal experimentation issues (N/A).

No issues regarding the use of human tissues.

No animal tissue or biological fluids issues.

No ionising radiation issues.

No primary data collection issues (N/A).

No underage/vulnerable people issues (UG).

No issues regarding existing/secondary data use (public anonymous and/or with explicit consent).

Note: Full compliance with the EU General Data Protection Regulation (GDPR) and the Data Protection Act (2018) is necessary for any personal data processing.

No controversial data issues.

No issues related to the collection of rare or protected plants.

No issues regarding the use of genetically modified (GM) plant material.

Instructions:

1. If the above feedback is **entirely** green then, based on your answers, there is **no need to apply for ethical approval** for your project.
2. If **any** part of the above feedback is **yellow/amber**, then there is at least one issue with your project that needs to be reviewed and **you must apply for ethical approval** to continue your project.
3. If **any** part of the above feedback is **red** then there is a serious ethical issue and **you cannot continue your project** as currently planned.

It is recommended that you print this Feedback Report to a PDF file for your records. You should also forward and discuss this Feedback Report PDF with your project supervisor. They will be able to advise if you have any further questions or if you need to apply for ethical approval.

Please review this report carefully to make sure that all your answers have been properly recorded.

*****Note that you can ignore all "white exclamation marks in red circles" in this document*****

1 Are you a student on a **final year undergraduate** programme, a **taught postgraduate** programme, or are you conducting a **research project**?

- (•) Final Year Undergraduate
- ( ) Taught Postgraduate
- ( ) Postgraduate Research Project
- ( ) Other Research Project

2 What is the **working title** of your project?

EnferVision – LIMS Bulk PRF Scanner/Process implemented as local web app using AI process

3 Who are the project **supervisors/advisors/principal investigators**?

Colm Dunphy / TBC

4 Does your project involve **human experimentation**?

( ) Yes (•) No

5 Does your project involve **live animal experimentation**?

( ) Yes (•) No

(6) Is the planned animal experimentation limited to **non-invasive procedures only** (such as feeding, weighing, or taking naturally voided faecal or hair samples), and does **not** involve any invasive procedures (such as taking rectal faecal samples or blood) from live animals?

( ) Yes ( ) No

7 Does your project involve the use of **human** remains/cadavers/tissues/cells/biological fluids/embryos/foetuses?

Yes No

(8) Do you intend to only use established **commercial human cell lines**, and no other **human** remains/cadavers/tissues/cells/biological fluids/embryos/foetuses in your project?

Yes No

9 Does your project involve the use of **animal cells, tissues** or **biological fluids**?

Yes No

(10) Do you intend to only use (1) **established commercial animal cell lines**, or (2) **slaughterhouse-derived tissues/fluids**, or (3) **fluids collected as part of routine animal husbandry** (e.g. milk) and no other animal tissues or biological fluids in your project?

Yes No

11 Does your project involve the **collection of rare or protected plants**?

Yes No

12 Does your project involve the generation or use of **genetically modified (GM) plant material**?

Yes No

(13) Do you agree to (1) only use **established genetically modified (GM) plant cell lines, seeds, or plant products** in your project, (2) **not generate new plant mutations** using chemical or other means, and (3) follow specified SETU **containment and use protocols** for GM plant materials at all times?

Yes No

14 Does your project involve the use of **ionising radiation**? (e.g. use of gamma ray spectrometry)

Yes No

(15) Do you agree to carefully **follow the instructions** of the SETU designated **Radiation Protection Officer (RPO)**, and **adhere to all legal requirements** as set out in the Radiological Protection Act 1991 (Ionising Radiation) Regulations (2019), regarding the use of ionising radiation materials and equipment?

Yes No

16 Does your project involve the **collection of any new (or primary) data** from **individual people or groups**?

Yes No

(17) Does your project involve the **collection of any new (or primary) individual or group data** that is **personally or uniquely identifying**? (e.g. data about people or organisations/companies/groups that could be used to identify those individuals or groups; data collection might take any form, including internet and social media data, etc.)

Yes No

(18) Will you ensure that participants who you are collecting data from are provided with **fair warning** and must provide **explicit informed consent** for any data collected?

Yes No

(19) Will you ensure that any project-related data collection, data storage, and data use is in **full compliance** with the **EU General Data Protection Regulation (GDPR)** and the **Data Protection Act (2018)**?

Yes No

(20) Does any of the data that you intend to collect include **sensitive or private personal information** about individuals, or **commercially sensitive information** about organisations/companies/groups?

Yes No

21 Does your project involve **persons under the age of 18 years** (i.e. minors), or **any vulnerable groups**? (e.g. prisoners, refugees, those in care, addiction service users, etc.)

Yes No

22 Does your project involve the use of **existing (or secondary) human data**? (i.e. data originally collected for another purpose)

Yes No

23 Is the existing or secondary human data you intend to use either (1) **anonymous/non-personally identifying** and in the **public domain**, or (2) available with **explicit and specific informed consent or permission** for the data to be **legally** reused in the way you intend?

Yes No

24 Are any aspects of the primary/secondary data you intend to use for the project **controversial** in nature?

Yes No

25 Before you submit the Ethics Checklist, you must **confirm all of the following**:

I understand that the Ethics Checklist is a formal declaration.
I have answered all questions on the Ethics Checklist carefully and truthfully.
The supervisor/advisor (or principal investigator) for the project is present as the Ethics Checklist is being submitted, or they have given me explicit permission to submit it in their absence.
I have had adequate ethics training and/or instruction prior to completing the Ethics Checklist.
I understand, and agree to abide by, the general ethical principle of "do no harm" for this project.
I will follow the instructions given in the Feedback Report.

26 Authentication Code (ask your project supervisor/advisor for this code)

Enter Student Number:

Enter the Authentication Code below and click "Verify Code" Verify Code

Note: If an INVALID authentication code is used then this submission is NULL and VOID

7471

Appendix II – Word Count

Total word count: 8596 (within 8,000 ± 10% limit). Breakdown:

Section	Count
Declaration	86
Titles	
Business Title	1
Academic Title	14
Abstract	324
Table of Contents	344
Table of Figures	274
1. Introduction	
1.1 Background	226
1.2 Problem Definition	60
1.3 Proposed Solution	112
1.4 Project Goals	119
2. Research and Analysis	
2.1 Data Characteristics and Constraints	185
2.2 Evaluation of Document Processing Approaches	614
2.3 Security, Privacy, and Compliance Considerations	159
2.4 Output Requirements	241
2.5 Audit and Traceability Requirements	177
2.6 Technology Stack Considerations	501
3. Modelling	
3.1 Process Workflow	222
3.2 Front-end	330
3.3 Back-end	109
4. Planning	
4.1 Front-end	
4.1.1 Authentication	114
4.1.2 Core UI Features	195
4.1.3 Access Control	106
4.1.4 User Interface Testing	69
4.2 Back-end	
4.2.1 Authentication	89
4.2.2 Upload and Ingestion	65
4.2.3 AI Data Extraction	56
4.2.4 Confirmation and Output	62
4.2.5 Auditing Functionality	81
4.2.6 Persistence Features	78
4.2.7 Back-end Testing	41
4.2.8 Action Tracking	35

Appendix III – OpenAI Parsing Script Request

The following is the prompt and AI response for an OpenAI parsing script for the project. This script was ultimately removed following a move to a full API call approach.

Prompt:

I have the following python script which uses PaddleOCR to identify and extract data from a label on a PRF:

```
from pathlib import Path
import json
import cv2
from paddleocr import PaddleOCR

inputFolder = Path("processing/toBeProcessed")
outputFolder = Path("processing/ocrTextOutput")

ROI_W = 0.4
ROI_H = 0.4

ocr = PaddleOCR(
    use_doc_orientation_classify=False,
    use_doc_unwarping=False,
    use_textline_orientation=False
)

def crop_label_roi(image):
    h, w = image.shape[:2]
    return image[0:int(h * ROI_H), 0:int(w * ROI_W)]

def extract_label_lines(result):
    label_lines = []
    for res in result:
        for block in res.get("rec_texts", []):
            label_lines.append(block)
    return label_lines

def main():
    for image_path in inputFolder.glob("*.jpg"):
        image = cv2.imread(str(image_path))
        if image is None:
            continue

        roi = crop_label_roi(image)

        result = ocr.predict(input=roi)
```

```
        label_lines = extract_label_lines(result)

        data = {
            "barcode": image_path.stem,
            "rawLines": label_lines
        }

        out_path = outputFolder / f"{image_path.stem}.json"
        out_path.write_text(json.dumps(data, indent=2), encoding="utf-8")

if __name__ == "__main__":
    main()
```

The next step in this pipeline is to parse the data contained in the .json file created for each PRF and combine it with the data contained in a .json file created following an API call, to create a combined .json file for downstream processing. Files can be matched based on filename as these will be matched to the PRF barcode. An example .json created following the API call, which contains the identified field entry alongside a confidence level, is as follows:


```
{
  "fasting": "Y",
  "fastingConfidence": 0.99,
  "pregnant": "",
  "pregnantConfidence": 0.0,
  "immunocomp": "",
  "immunocompConfidence": 0.0,
  "datesample": "12/02/20",
  "datesampleConfidence": 0.8,
  "timesample": "09:15",
  "timesampleConfidence": 0.8,
  "urgency": "TAT",
  "urgencyConfidence": 0.95,
  "extrefno": "OLHD12354",
  "extrefnoConfidence": 0.99,
  "clindetails": "",
  "clindetailsConfidence": 0.0,
  "gpName": "DR. SHANE GLEESON",
  "gpNameConfidence": 0.99,
  "practice": "CARRICK ROAD MEDICAL CENTRE LIS NA DARA DUNDALK G.M.S.
NUMBER 75985",
  "practiceConfidence": 0.99,
  "barcode": "OLHD12354"
}
```


An example of the fields and data that would need to be returned from the parsing script would be as follows:

```
{
  "barcode": "OLHD09876",
  "patientFN": "Sherlock",
  "patientFNConfidence": 0.97,
  "patientSN": "Holmes",
  "patientSNConfidence": 0.96,
  "dob": "01/01/1956",
  "dobConfidence": 0.99,
  "gender": "M",
  "genderConfidence": 0.98,
  "address1": "221B Baker St",
  "address1Confidence": 0.91,
  "address2": "",
  "address2Confidence": 0.0,
  "county": "Dublin",
  "countyConfidence": 0.95,
  "eircode": "",
  "eircodeConfidence": 0.0
}
```


The combination of the 2 files would then look as follows:


```
{
  "barcode": "OLHD09876",
  "patientFN": "Sherlock",
  "patientFNConfidence": 0.97,
  "patientSN": "Holmes",
  "patientSNConfidence": 0.96,
  "dob": "01/01/1956",
  "dobConfidence": 0.99,
  "gender": "M",
  "genderConfidence": 0.98,
  "address1": "221B Baker St",
  "address1Confidence": 0.91,
  "address2": "",
  "address2Confidence": 0.0,
  "county": "Dublin",
  "countyConfidence": 0.95,
  "eircode": "",
  "eircodeConfidence": 0.0,
  "fasting": "Y",
  "fastingConfidence": 0.88,
  "pregnant": "N/A",
  "pregnantConfidence": 0.99,
  "immunocomp": "N",
  "immunocompConfidence": 0.83,
  "datesample": "15/03/2026",
  "datesampleConfidence": 0.93,
  "timesample": "09:25",
  "timesampleConfidence": 0.89,
```

```
  "urgency": "TAT",
  "urgencyConfidence": 0.86,
  "extrefno": "123456",
  "extrefnoConfidence": 0.81,
  "clindetails": "Patient exhibits LBP",
  "clindetailsConfidence": 0.76,
  "gpName": "Dr Watson",
  "gpNameConfidence": 0.84,
  "practice": "Carrick Road Medical Centre, Lis Na Dara, Dundalk",
  "practiceConfidence": 0.79
}
```


Confidence levels produced from the parser can, for now, be based on parsing confidence or randomly generated each time as this is merely for proof of concept of a potential pipeline. Future iterations will apply a document processing service that will base confidence levels on a combination of extraction and parsing. Four example extracted .json files, which contain quite a bit of noise, are as follows:


```
{
  "barcode": "OLHD12354",
  "rawLines": [
    "LF-GEN-0032 Rev.No.5",
    "OUR LADYOF LOU",
    "BLO",
    "ne",
    "Sherlock Holmes",
    "DOB:01/01/1956",
    "Gender:M",
    "Pul",
    "Address:221BBaker St,",
    "Pri",
    "Dublin"
  ]
}

{
  "barcode": "OLHD978657",
  "rawLines": [
    "LF-GEN-0032 Rev.No.5",
    "OUR LADYOF LOL",
    "BLC",
    "AaronAAaronson",
    "ne",
    "01/03/2008",
    "11/03/202610:10",
    "PL",
    "123 AISLINGPARK,",
    "DUNDALK,CO.LOUTH",
    "Pr"
```

```
  ]
}

{
  "barcode": "SVUH12364",
  "rawLines": [
    "AE",
    "St. Vincent's U",
    "St.Vin",
    "CRASAAAE",
    "Bradshaw's Lane",
    "Surgery John Watson",
    "DOB:01/01/1934",
    "F",
    "Gender:M",
    "RRRESR",
    "Pi",
    "Address: Appledore, 123",
    "G",
    "Lane, Wicklow",
    "S",
    "PatientPhone No:",
    "P"
  ]
}

{
  "barcode": "SVUH978667",
  "rawLines": [
    "",
    "St.Vin",
    "JonathanSWIFT",
    "36 Rory Doyle Park, Dun Laoghaire,",
    "Surname",
    "Co.Dublin",
    "Forenames",
    "DOB:27/05/2001",
    "Date:11/03/2026",
    "Gender: Female",
    "Dr Ross Doyle",
    "PatientAdo",
    "GMS:4P67986R",
    "Monkstown Practice",
    "ame",
    "SVUH GP",
    "Patient Phone No:",
    "Phone No"
  ]
}
```

Response:

Here's a solid proof-of-concept parser/merger script for the next stage of your pipeline.

It does three things:

1. reads the OCR .json
2. parses patient/address fields from noisy rawLines
3. merges that with the API .json using the barcode/filename

```
from pathlib import Path
import json
import re
import random

# ----------------------------
# Folders
# ----------------------------
ocrFolder = Path("processing/ocrTextOutput")          # input from PaddleOCR stage
apiFolder = Path("processing/apiJsonOutput")          # input from API stage
combinedFolder = Path("processing/combinedOutput")    # final merged output

combinedFolder.mkdir(parents=True, exist_ok=True)

# ----------------------------
# Helpers
# ----------------------------
COUNTIES = [
   "ANTRIM", "ARMAGH", "CARLOW", "CAVAN", "CLARE", "CORK", "DERRY",
"DONEGAL",
   "DOWN", "DUBLIN", "FERMANAGH", "GALWAY", "KERRY", "KILDARE",
"KILKENNY",
   "LAOIS", "LEITRIM", "LIMERICK", "LONGFORD", "LOUTH", "MAYO", "MEATH",
   "MONAGHAN", "OFFALY", "ROSCOMMON", "SLIGO", "TIPPERARY", "TYRONE",
   "WATERFORD", "WESTMEATH", "WEXFORD", "WICKLOW"
]

NOISE_PATTERNS = [
   r"LF-GEN-\d+",
   r"REV\.?NO\.?\d+",
   r"OUR LADY",
   r"ST\.?\s*VINCENT",
   r"ST\.?\s*VIN",
   r"BLO",
   r"BLC",
   r"PL$",
   r"PI$",
   r"PR$",
   r"PUL$",
   r"AE$",
```

```
    r"CRASAAAE",
    r"RRRESR",
    r"PHONE NO",
    r"PATIENTPHONE",
    r"PATIENTADO",
    r"SURNAME",
    r"FORENAMES",
    r"GMS:",
    r"SVUH GP",
    r"NAME$",
    r"NE$",
]

def rand_conf(low=0.75, high=0.99):
    return round(random.uniform(low, high), 2)

def empty_field():
    return "", 0.0

def clean_line(line):
    if not isinstance(line, str):
        return ""
    line = line.strip()
    line = re.sub(r"\s+", " ", line)
    return line

def upper(line):
    return clean_line(line).upper()

def is_noise(line):
    u = upper(line)
    if not u:
        return True
    for pattern in NOISE_PATTERNS:
        if re.search(pattern, u):
            return True
    return False

def normalize_date(date_str):
    date_str = date_str.strip()
    m = re.match(r"^(\d{2})/(\d{2})/(\d{2})$", date_str)
    if m:
        dd, mm, yy = m.groups()
        yyyy = "20" + yy if int(yy) <= 30 else "19" + yy
        return f"{dd}/{mm}/{yyyy}"
    return date_str

def extract_date(line):
    m = re.search(r"(\d{2}/\d{2}/\d{2,4})", line)
    if m:
```

```
        return normalize_date(m.group(1))
    return None

def extract_gender(line):
    u = upper(line)

    if "GENDER" in u:
        if "FEMALE" in u:
            return "F"
        if "MALE" in u:
            return "M"
        m = re.search(r"GENDER\s*[:\-]?\s*([MF])\b", u)
        if m:
            return m.group(1)

    if u in {"M", "F"}:
        return u

    return None

def looks_like_name(line):
    if not line or is_noise(line):
        return False

    if extract_date(line) or extract_gender(line):
        return False

    u = upper(line)

    if "ADDRESS" in u or "DATE:" in u or "DOB" in u or "GMS:" in u:
        return False

    # Must contain letters
    if not re.search(r"[A-Za-z]", line):
        return False

    # Avoid likely address lines
    if re.search(r"\d", line) and any(tok in u for tok in ["PARK", "ST", "LANE", "ROAD",
"AVE", "AVENUE", "DRIVE", "COURT"]):
        return False

    return True

def split_name(full_name):
    parts = full_name.strip().split()
    if len(parts) == 1:
        return parts[0].title(), ""
    return parts[0].title(), " ".join(parts[1:]).title()

def maybe_fix_joined_name(line):
```

```
    # Example: AaronAAaronson -> Aaron Aaronson
    # Example: JonathanSWIFT -> Jonathan SWIFT
    fixed = re.sub(r"([a-z])([A-Z])", r"\1 \2", line)
    fixed = re.sub(r"([A-Z]{2,})([A-Z][a-z])", r"\1 \2", fixed)
    return fixed.strip()

def extract_name(lines):
    candidates = []

    for line in lines:
        raw = clean_line(line)
        fixed = maybe_fix_joined_name(raw)

        if looks_like_name(fixed):
            candidates.append(fixed)

    # Prefer likely patient names over doctor lines
    filtered = []
    for c in candidates:
        uc = upper(c)
        if uc.startswith("DR ") or uc.startswith("DR. ") or "SURGERY" in uc:
            continue
        filtered.append(c)

    if filtered:
        return filtered[0]

    # fallback: if all candidates were filtered, use first candidate
    if candidates:
        return candidates[0]

    return ""

def extract_dob(lines):
    for line in lines:
        u = upper(line)
        if "DOB" in u:
            d = extract_date(line)
            if d:
                return d

    # fallback: first date that is not obviously sample date
    for line in lines:
        u = upper(line)
        if "DATE:" in u:
            continue
        d = extract_date(line)
        if d:
            return d
```

```
    return ""

def extract_gender_field(lines):
    for line in lines:
        g = extract_gender(line)
        if g:
            return g
    return ""

def extract_address(lines):
    address1 = ""
    address2 = ""
    county = ""
    eircode = ""

    cleaned = [clean_line(x) for x in lines if clean_line(x)]

    # 1. Look for explicit Address:
    for i, line in enumerate(cleaned):
        u = upper(line)
        if "ADDRESS:" in u:
            value = re.sub(r"(?i)^.*ADDRESS\s*:\s*", "", line).strip(" ,")
            if value:
                address1 = value

            # try next line as address2/county
            if i + 1 < len(cleaned):
                nxt = cleaned[i + 1].strip(" ,")
                nu = upper(nxt)
                county_match = re.search(r"\bCO\.?\s*([A-Z]+)\b", nu)
                if county_match:
                    county = county_match.group(1).title()
                    rest = re.sub(r"\bCO\.?\s*[A-Z]+\b", "", nxt, flags=re.I).strip(" ,")
                    if rest:
                        address2 = rest
                elif upper(nxt) in COUNTIES:
                    county = nxt.title()
                else:
                    address2 = nxt

            if i + 2 < len(cleaned) and not county:
                nxt2 = cleaned[i + 2].strip(" ,")
                if upper(nxt2) in COUNTIES:
                    county = nxt2.title()

            break

    # 2. Fallback: find address-looking lines
    if not address1:
        for i, line in enumerate(cleaned):
```

```
            u = upper(line)

            if "ADDRESS" in u:
                continue

            # starts with a number, common for address
            if re.match(r"^\d+[A-Za-z]?\s+", line):
                address1 = line.strip(" ,")
                if i + 1 < len(cleaned):
                    nxt = cleaned[i + 1].strip(" ,")
                    nu = upper(nxt)

                    county_match = re.search(r"\bCO\.?\s*([A-Z]+)\b", nu)
                    if county_match:
                        county = county_match.group(1).title()
                        rest = re.sub(r"\bCO\.?\s*[A-Z]+\b", "", nxt, flags=re.I).strip(" ,")
                        if rest:
                            address2 = rest
                    elif upper(nxt) in COUNTIES:
                        county = nxt.title()
                    else:
                        address2 = nxt
                break

    # 3. County detection anywhere
    if not county:
        for line in cleaned:
            u = upper(line)
            m = re.search(r"\bCO\.?\s*([A-Z]+)\b", u)
            if m and m.group(1) in COUNTIES:
                county = m.group(1).title()
                break
            if u in COUNTIES:
                county = line.title()
                break

    # 4. Eircode detection
    # Irish Eircode format: A65 F4E2, D02 X285 etc.
    for line in cleaned:
        m = re.search(r"\b([A-Z]\d{2}|[A-Z]\d[A-Z]|[A-Z]{3})\s?[A-Z0-9]{4}\b", upper(line))
        if m:
            eircode = m.group(0).replace(" ", "")
            break

    return address1, address2, county, eircode

def parse_ocr_json(ocr_data):
    raw_lines = ocr_data.get("rawLines", [])
    barcode = ocr_data.get("barcode", "")
```

```
    full_name = extract_name(raw_lines)
    patient_fn, patient_sn = split_name(full_name) if full_name else ("", "")
    dob = extract_dob(raw_lines)
    gender = extract_gender_field(raw_lines)
    address1, address2, county, eircode = extract_address(raw_lines)

    parsed = {"barcode": barcode}

    parsed["patientFN"] = patient_fn
    parsed["patientFNConfidence"] = rand_conf() if patient_fn else 0.0

    parsed["patientSN"] = patient_sn
    parsed["patientSNConfidence"] = rand_conf() if patient_sn else 0.0

    parsed["dob"] = dob
    parsed["dobConfidence"] = rand_conf() if dob else 0.0

    parsed["gender"] = gender
    parsed["genderConfidence"] = rand_conf() if gender else 0.0

    parsed["address1"] = address1
    parsed["address1Confidence"] = rand_conf() if address1 else 0.0

    parsed["address2"] = address2
    parsed["address2Confidence"] = rand_conf() if address2 else 0.0

    parsed["county"] = county
    parsed["countyConfidence"] = rand_conf() if county else 0.0

    parsed["eircode"] = eircode
    parsed["eircodeConfidence"] = rand_conf() if eircode else 0.0

    return parsed

def merge_json(parsed_data, api_data):
    merged = dict(parsed_data)

    for key, value in api_data.items():
        if key == "barcode":
            continue
        merged[key] = value

    # Ensure barcode exists and matches filename-based workflow
    merged["barcode"] = parsed_data.get("barcode") or api_data.get("barcode", "")
    return merged

def main():
    for ocr_path in ocrFolder.glob("*.json"):
        barcode = ocr_path.stem
        api_path = apiFolder / f"{barcode}.json"
```

```
        if not api_path.exists():
            print(f"Skipping {barcode}: no matching API file found")
            continue

        with ocr_path.open("r", encoding="utf-8") as f:
            ocr_data = json.load(f)

        with api_path.open("r", encoding="utf-8") as f:
            api_data = json.load(f)

        parsed_data = parse_ocr_json(ocr_data)
        combined_data = merge_json(parsed_data, api_data)

        out_path = combinedFolder / f"{barcode}.json"
        with out_path.open("w", encoding="utf-8") as f:
            json.dump(combined_data, f, indent=2, ensure_ascii=False)

        print(f"Created {out_path}")

if __name__ == "__main__":
    main()
```

A few notes on how this works:

- It matches files by filename/stem, so OLHD12354.json from OCR is merged with OLHD12354.json from the API stage.
- Parser confidence is randomly generated for now, as requested.
- It handles some of the OCR noise in your examples:
    - DOB:01/01/1956
    - Gender:M
    - Gender: Female
    - joined names like JonathanSWIFT and AaronAAaronson
    - address lines like Address:221BBaker St,
    - county formats like Dublin or CO.LOUTH

A few realistic limitations in this proof-of-concept:

- split_name() assumes first token is forename and remaining tokens are surname. That works for your examples, but not all real names.
- AaronAAaronson may become Aaron Aaaronson depending on OCR weirdness. You may want custom cleanup rules later.
- Address parsing is heuristic-based, so it will need refinement as you collect more samples.

For your four examples, this should produce roughly:

- Sherlock Holmes → patientFN Sherlock, patientSN Holmes
- AaronAAaronson → likely patientFN Aaron, patientSN Aaaronson or similar
- Surgery John Watson is filtered as doctor-ish text, so this sample may rely more on fallback behaviour
- JonathanSWIFT → patientFN Jonathan, patientSN Swift

A stronger next iteration would be to score candidate lines instead of just taking the first plausible match. For example:

- lines near DOB get higher weight for patient info
- lines containing Dr get lower weight for patient name
- address lines containing digits and street words get higher weight

If you want, I can also give you a version that:

- writes an intermediate parsedOutput file before merging, or
- uses deterministic parser confidence instead of random values.